

Models as Parametric Kernels

An Order, an Operator and a Polynomial:

Deriving the Facts a Model Governance System Otherwise Declares

Ashutosh Sinha

Independent Researcher

`ajsinha@gmail.com`

Abstract

A large regulated institution operates thousands of quantitative artefacts — closed-form pricers, market-calibrated surfaces, estimated scorecards, learned classifiers, online-adaptive systems, foundation-model applications, vendor black boxes, elicited rating schemes, deterministic rule sets — and must govern them as one population. The systems built to discharge that obligation run on *declared* facts: somebody types what kind of artefact this is, somebody asserts that this version may replace that one, somebody writes down what a training set could have known, somebody records in prose what a conclusion rests on. A declared fact is true at the moment it is typed and at no guaranteed moment afterwards, and every pathology practitioners report in these systems is a declaration that has drifted from the thing it describes.

We argue that four of these facts need not be declared at all, and give the algebra that derives each. A model is a morphism in **Para(Stoch)**: a parameter object P and a kernel $f: P \otimes X \rightarrow \mathcal{D}(Y)$. From *how P is inhabited* we derive the trainability class, so that “some governed artefacts are never trained” becomes a type-level fact rather than a special case to be remembered. From a single partial order on schemas — $A \sqsubseteq B$, read *A can stand in for B* — we derive substitutability, feature-set adequacy and whether two artefacts may be wired together; the order is a lattice, its meet is partial and informatively so, and one relation replaces four implementations that were free to disagree. From the point-in-time read presented as an operator $\text{AsOf}(R, \ell, a)$ we derive reproducibility: the operator is idempotent, commutes with projection, monotone in the observation time and — because the ingest bound is $\min(\ell, a)$ rather than a — *saturating* at the label, which is precisely the guarantee that a training row re-assembled a year later is the row that was assembled originally. From the free commutative semiring $\mathbb{N}[X]$ we derive every question about what a claim or a feature rests on, each as a pushforward along a homomorphism; the ingest clock of a derived feature is one of them, which upgrades a rule that could be forgotten into a homomorphism that has no exceptions.

We are equally explicit about what does not derive. Aggregate risk cannot be compositional, and the obstruction is exactly the comonoid copy map — structural, rather than a modelling difficulty a better correlation assumption removes. A currency valuation over a totally ordered carrier is not a semiring valuation at all, because its zero does not annihilate, and it becomes one only when “no support” is given a value of its own. And some facts *constitute* the standard they would be checked against: a tiering rule, a regime encoding, an approval. That boundary, drawn for governance reasons, turns out to be the same boundary that decides where machine generation is admissible, which we regard as the most useful consequence of the account.

Finally, we extend the account to the judgements a model-risk function makes day to day — materiality, periodic review, monitoring, the replacement of one model by another, fairness, and the governance of black boxes and of applications built on large language models — and show that each is a declared rule applied to derived facts. The properties that make them trustworthy are small and exact: the derived tier is monotone; a paired comparison between two models is licensed precisely by equal hashes of their escrowed

holdouts; a fairness criterion over error sizes alone is blind to the direction of the error; and evidence for an application whose parameter object is authored rather than fitted is bound, by hash, to its definition and to the evaluation it was gathered on.

Every formal claim is marked with what carries it in MAYA, a platform the author built to the account and released as the artefact accompanying this paper (<https://github.com/ajsinha/maya>). The markers are not uniformly favourable and are not meant to be: several constructions are implemented, several run in a weaker form the paper states exactly, and several are not implemented at all. Section 10 gives the register, Section 12 gives the system and its measurements, and Section 14 states what the implementation does not establish — which includes the claim that most concerns a practitioner.

1 Introduction

1.1 The domain

Consider the population of quantitative artefacts operated by a large financial institution. It includes a Black–Scholes implementation whose parameters come from financial theory and are never estimated from data; a stochastic volatility surface re-solved against market quotes every morning; a logistic scorecard refitted annually on a historical sample; a gradient-boosted classifier retrained weekly on transaction streams; an adaptive threshold that updates itself continuously in production; a retrieval-augmented language model drafting regulatory narratives; a third-party credit score whose internals are contractually unavailable; a country-risk rating scheme whose weights are set by a committee; and several thousand rule sets and spreadsheets.

These artefacts have essentially nothing in common at the level of implementation. Yet they are subject to a single normative obligation. Supervisory guidance requires an institution to maintain an inventory of them, classify each by materiality, subject each to independent challenge proportionate to that classification, monitor performance, document design and limitations, control change, and assess risk both individually and *in aggregate*, accounting for “interactions and dependencies among models; reliance on common assumptions, data, or methodologies” [40, 41]. Different jurisdictions define the governed population differently: one regime narrows it to complex quantitative methods applying statistical, economic or financial theory, explicitly excluding deterministic rules and, remarkably, generative systems; another defines it broadly enough to include expert-judgment inputs and qualitative outputs [41, 42]. An institution operating in both must satisfy a narrow and a broad scope simultaneously over one population.

Remark 1.1 (On the supervisory sources, and how much this paper leans on them). We cite two supervisory texts throughout: the United Kingdom’s SS1/23 [41], in force since 2024, and a 2026 United States revision of the model risk management guidance, cited as SR 26-2 / OCC Bulletin 2026-13 [40]. A reader should treat the second differently from the first. It is recent, and its status and final text may not be settled in the reader’s jurisdiction or at the reader’s date.

It matters less to the argument than it may appear. The four declared-fact pathologies below are readable off SR 11-7 and SS1/23 alone: SR 11-7 has carried the aggregate-risk expectation, the independent-challenge requirement and the inventory obligation since 2011 [39]. And the definitional narrowing we use to motivate plural scope is a *recurring* feature of model-risk definitions rather than a property of one revision — SR 11-7’s own “quantitative method . . . that applies statistical, economic, financial, or mathematical theories” already sits awkwardly against a deterministic rule set that a firm nonetheless has to govern, and against a language model that governs nothing and drafts everything.

Where a claim here depends specifically on the 2026 revision it is doing *illustrative* and not load-bearing work, and where the reader’s jurisdiction has a different text the substitution is mechanical. That it is mechanical is the point of Section 8: a regime is an institution,

and swapping one for another is a change of signature and sentences rather than a change of platform. A formalism whose argument collapsed when a supervisor revised a definition would be evidence against itself.

1.2 Declared facts, and why they rot

The systems built for this task fail in recognisable ways. It is usual to present those failures as four pathologies. We prefer a sharper diagnosis, because it points at the remedy: in each case a *fact that could have been derived from structure was instead declared by a person*, and a declared fact is only true at the moment it is typed.

D1: kind is declared. A field records that this artefact is “an ML model” or “a pricer” or “other”. Downstream, evidence requirements switch on that field, so a closed-form pricer is asked for its training dataset. That is not a missing value; it is a type error, and a register that cannot say so accumulates records that mean nothing. The declaration is also the only thing standing between the taxonomy and reality: nothing checks that an artefact declared “rule set” has no fitted parameters.

D2: fit is declared. “This version may replace that one.” “This feature set provides what that model reads.” “This model’s output goes into that model.” Each is a claim that one thing can stand where another stood. In the systems we have examined each is answered by its own code path, and four implementations of one relation eventually disagree — predictably in the direction of permitting more, because that is the direction in which nobody files a bug.

D3: currency is declared. The condition under which a training row is admissible is written into a query by whoever wrote the query. Whether the query implements the condition is not checked, and the failure — a fact used before it was knowable — is invisible to every performance metric, because it improves them.

D4: support is declared. An evidence log records, in prose, that a version was validated. Nothing in the record establishes what the conclusion rests on, so nothing can compute whether a cited set suffices, what the cheapest route to closing a gap is, or whether a document is stale. The claim is unfalsifiable, which is the opposite of what assurance requires.

1.3 The thesis

Each of D1–D4 is a fact about structure that has been recorded as a fact about opinion. Our thesis is that all four derive, and that the algebra required is small: a definition, an order, an operator and a polynomial.

1. Section 3 defines a model as a morphism of **Para(Stoch)** and derives the trainability class from how the parameter object is inhabited (Theorem 3.3). D1 dissolves: training is one way of inhabiting P , not part of being a model.
2. Section 4 gives *one* partial order on schemas, $A \sqsubseteq B$ read as “ A can stand in for B ”, shows it is a lattice with partial meet and empty top (Theorems 4.6 and 4.8), and shows that the four questions of D2 are that one comparison (Theorem 4.9).
3. Section 5 makes model-to-model composition a type-check rather than a drawing (Theorem 5.1), and then proves the negative result that bounds it: no aggregate risk assignment is both compositional and sensitive to shared dependency, with the comonoid copy map as the exact obstruction (Theorem 5.6).

4. Section 6 presents the point-in-time read as an operator and proves four properties of it, of which the fourth — saturation at the label — *is* reproducibility (Theorems 6.2 and 6.4). D3 dissolves: the condition is a property of an operator rather than a convention in a query. Section 6.3 then relates the operator to the arrangement MAYA implements — a scalar cut and a signed certificate of the per-row bound — and proves what that arrangement is worth (Theorems 6.7 and 6.8).
5. Section 7 annotates evidence and derived features over the free commutative semiring $\mathbb{N}[X]$ and shows that every other question is a pushforward along a homomorphism (Theorem 7.1), including the ingest clock of a derived feature (Theorem 7.3). D4 dissolves. We also prove that a currency valuation cannot be one of these homomorphisms over its own carrier (Theorem 7.5), give the repair (Theorem 7.6), and show the defect the repair addresses arising in three separate computations of the implementation (Section 7.4).
6. Section 3.4 indexes evidence requirements as the fibres of a fibration, and shows that the base of that fibration must itself be derived: a declared base cannot deliver conservative extension and totality together (Theorem 3.7). Section 3.5 treats the one class whose parameter object has logical rather than numerical structure, and states exactly what becomes decidable there and what does not (Theorem 3.8).
7. Section 8 draws the complement honestly: the facts that must be declared because they *constitute* the standard against which anything else is checked.
8. Section 9 applies that frame to the governance layer itself — materiality, review, monitoring, champion and challenger, fairness, black boxes and language-model applications — and proves the properties each needs (Theorems 9.2, 9.5, 9.7 and 9.9).
9. Section 10 states the discipline that keeps the rest from becoming ornament, gives the register of every claim against the implementation — what runs, what runs in part, what is not there — and states the properties an executable law delivers that a documented one does not.
10. Section 11 observes that the derive/declare boundary of Sections 4 and 6 to 8 is exactly the boundary that decides where machine-generated output may be accepted (Theorems 7.7 and 11.2).

1.4 What is borrowed and what is new

We are explicit, because the value of an integrative paper depends on it. Markov categories are due to Fritz [1] and, in the string-diagrammatic form we use, to Cho and Jacobs [2]; the underlying monad is Giry’s [3]. The **Para** construction is due to Fong, Spivak and Tuyeras [4], elaborated by Cruttwell, Gavranović, Ghani, Wilson and Zanasi [5] and by Capucci and collaborators [6]. Lattice theory is standard [7, 8], as is the variance discipline our order recovers [9, 10]. Provenance semirings and the universality of $\mathbb{N}[X]$ are due to Green, Karvounarakis and Tannen [11, 12], with the semiring hierarchy of [13, 14]. Bitemporal data management is Snodgrass’s [16], and reached the standard in SQL:2011 [17]. Assume–guarantee contracts are due to Benveniste and collaborators [22]; institutions to Goguen and Burstall [20]; sound abstraction to Cousot and Cousot [24]; well-behaved lenses to Foster and collaborators [25]; property-based testing to Claessen and Hughes [26]. None of these is our contribution.

What we claim is new, in decreasing order of confidence: (i) the derivation of the trainability classification from the inhabitation of P (Theorem 3.3), and the observation that it makes “never trained” a type and not an exception; (ii) the identification of *one* schema order as the common content of substitutability, feature-set adequacy and composability, together with the informative partiality of its meet (Theorems 4.8 and 4.9); (iii) the presentation of the

point-in-time read as an operator and the saturation law as the reproducibility guarantee (Theorems 6.2 and 6.4) — the point-in-time *condition* is folklore, and what appears not to be stated anywhere is that the correct ingest bound is $\min(\ell, a)$ and that this is what makes the read stable; (iv) the diagnosis in Theorem 5.6 of the copy map as the obstruction to compositional risk. We should be careful about what is new here and what is not. That aggregate model risk is not additive, that shared data, shared assumptions and shared methodology create concentration, and that model interdependency needs its own analysis are all established in practice and in the supervisory literature [40, 41], and dependency-aware aggregation has a substantial quantitative-risk history. What we claim is the *diagnosis*: that the obstruction is not a modelling difficulty to be approximated away but the comonoid copy map, that it is therefore structural, and that this yields a sharp statement of what a correct aggregate cannot be; (v) the application of semiring provenance to assurance evidence and, one layer across, to derived features, so that a data-engineering rule becomes a homomorphism (Theorem 7.3); (vi) Theorem 7.5, an elementary negative result we record because the corresponding positive claim is easy to reach for and, as Section 7.4 shows, easy to build three times without noticing; (vii) Theorem 3.7, which we believe to be new and which is the paper’s own thesis applied to its own machinery: an indexed family of governance obligations cannot be both freely extensible and provably total unless its index is computed rather than recorded; (viii) the observation that the derive/declare boundary and the oracle boundary coincide (Section 11); (ix) in Section 9, two elementary results we record because practice routinely violates them: that a fairness criterion over error sizes alone cannot see direction (Theorem 9.7), and that evidence for an authored parameter object must be bound to its definition and its evaluation by hash (Theorem 9.9).

1.5 How to read this paper, and how to check it

Section 2 states the objects in engineering terms before any of the mathematics begins, because the constructions below are worth little if what they quantify over is only sketched. A reader who already builds this kind of system can skim it and should still read the paragraphs on the two clocks and on pins.

The mathematics is elementary; the vocabulary is not evenly distributed across the audiences this work addresses. Every formal statement is therefore followed by a shaded *plain terms* gloss, and where it earns one, a worked example from banking. A reader who wants the argument without the notation can follow the shaded material alone and lose nothing essential.

Every formal claim carries one of four markers. **Runs** names the module and the test in the MAYA repository that carry it, as paths from the repository root. **In part** says which part runs and which does not. **Not executed** says the claim is implemented and no test asserts it. **Not in MAYA** says the system does not carry the construction at all.

The last two matter more than the first. A paper is a declaration, and this one argues that declarations decay; a paper claiming an implementation it does not exhibit, or implying that every claim is checked when some are not, would be committing in miniature the failure it is about. So the markers are placed where a reader would look for the favourable one, and Section 10 collects them into a register that can be read against the repository in an afternoon. The register is not flattering: of its rows, a minority run, several run in a weaker form that is stated exactly, and several are mathematics with no executable counterpart at all.

2 The objects, in engineering terms

The mathematics begins in Section 3. This section is what it is about. A reader who has built data infrastructure will recognise most of it and should still read the four paragraphs on clocks and pins, because the paper’s later guarantees are statements about exactly those two constructions and are easy to mishear as statements about ordinary versioning. A reader who

has built model risk systems will recognise the vocabulary of warrants and will find the input side more specific than is usual.

Nothing here is a contribution. It is the setting, written in the words a system that implements it has to use, and it is written first because the constructions in the rest of the paper are worth very little if the objects they quantify over are only sketched.

A model is a compute kernel with dials. A thing that takes inputs and produces an output, in a way governed by numbers that are not themselves inputs. Black–Scholes takes spot, strike, tenor, rate and volatility and returns a price: the formula comes from theory and there are no dials at all. A credit scorecard takes eight attributes of an applicant and returns a probability: the same shape, and forty-two dials that somebody had to set. The distinction between what goes in and what was set is the one this paper leans on hardest, and it is the distinction most registers do not record, because a column called `inputs` that holds both loses it.

The dials are set in one of several ways, and training is one of them. Theory supplies them and they never move. A calibration solves them against this morning’s quotes. An estimator fits them to a historical sample. A training run learns them from data and may keep learning in production. A committee agrees them. An author writes them down as rules. These are not different kinds of model; they are different ways of inhabiting the same slot, which is what Section 3 makes precise. A system whose taxonomy is “model” versus “rule set” versus “ML model” has encoded the method of setting the dials into the identity of the thing, and then has to maintain that encoding by hand.

A feature is not a column in somebody’s table. It is a named, typed, versioned definition of a quantity, together with the data that has ever been recorded for it. The definition says what the quantity is, what it is indexed by, where its values come from, and what may be done about gaps. The data is an append-only log: a correction is a new record, never an overwrite. That last property is not housekeeping. It is the precondition for everything in Section 6, and a store that overwrites cannot support the guarantee however carefully it is queried.

Every value carries two clocks. The *event time* is the moment the value describes: the quarter these accounts cover, the day this price closed. The *knowledge time* is the moment the recording system learnt it. They are independent, and the gap between them is where the failure lives. A borrower files accounts dated 31 March; they arrive on 20 May; in August the borrower restates them downward. Both records are true, both belong in the store, and they differ only in knowledge time. A system that keeps one clock cannot tell them apart, and will hand a model building a training row for a decision taken in June the restated figure — a number that exists *because* the default it is being asked to predict already happened. The model tests beautifully and fails in production, and the post-mortem blames drift.

A feature set is what a model is actually defined over. Features are defined separately and are useful separately; a model reads several of them together, aligned on a shared index. A feature set names that bundle: which features fill which slots, how they are aligned, what to do where one has a value and another does not. The schema — the named, typed slots — is separate from what fills them, and the separation earns its keep. Changing which feature fills a slot does not change the space the model is defined over; it changes what the model was fitted on, which is a different event deserving a different control.

A pin freezes what the data was, and is named by it. A version of a definition says how a thing is assembled. A pin says what it assembled: the resolved rows, sealed, so that the same read a year later returns the same bytes. Crucially the pin’s identity *is* a hash of its own contents, so two pins of the same data are the same pin and changed data cannot hide behind an unchanged name. A feature set is pinned as a whole or not at all: a bundle in which three members are frozen and one is still moving is not a frozen input, and the arrangement that admits it will be discovered a year later by somebody who cannot reproduce a number.

Training, precisely. Choosing the dials from data. It needs three things, and a system that supplies fewer is not governing it: a *frozen input*, so that the run can be repeated — which is what the pin is for; a *target*, the quantity being predicted, which must not also be reachable through the inputs; and a *record* of which model version, which input, which target and whose authority, which is what the next paragraph is.

A warrant is a licence to compute, not a note about a computation. A training warrant binds one model version to one pinned input and one target, and receives the parameters the run produced. An execution warrant licenses a model version, its parameters and its input contract to be run, under conditions that can be checked while it runs, and can be suspended. Both are permissions with a holder, a scope and an expiry. The distinction from a log entry is the whole point: a log says what happened and a warrant says what may happen, so a warrant can be refused before the fact, which is the only moment at which a governance system can prevent anything.

In plain terms. Six words carry the rest of this paper. A *feature* is a defined quantity and everything ever recorded for it, each value stamped with when it was true and when you found out. A *feature set* bundles features into the thing a model reads. A *pin* freezes what a feature or a set held, and is named by a hash of that content. *Parameters* are the dials; *training* is one way of setting them, from data, against a frozen input. A *warrant* is permission to fit or to run, granted in advance and refusable.

The argument of the paper is that four facts a register normally asks somebody to type — what kind of thing this is, whether this can stand in for that, what a training row could have known, and what a conclusion rests on — are consequences of these objects, and can be computed from them.

3 The kernel, and what the parameter object decides

3.1 Ambient setting

We work in a Markov category: a symmetric monoidal category $(\mathcal{C}, \otimes, I)$ in which the unit I is terminal and every object X carries a commutative comonoid structure $\text{copy}_X: X \rightarrow X \otimes X$, $\text{del}_X: X \rightarrow I$, compatible with the monoidal product and with del natural [1, 2]. Morphisms behave as Markov kernels; the canonical example is **Stoch**, the Kleisli category of the Girmonad on measurable spaces [3]. A morphism f is *deterministic* when $\text{copy}_Y \circ f = (f \otimes f) \circ \text{copy}_X$.

Two structural features earn their place immediately. Terminality of I says every morphism may be discarded — outputs need not be used, which is what makes an artefact’s *exposure* a meaningful separate quantity. Explicit copying says an output may be consumed by several downstream artefacts, which, as Section 5 shows, is exactly the structure responsible for non-compositional aggregate risk. A Cartesian category has both properties implicitly and therefore cannot make either visible.

3.2 Definition

Definition 3.1 (Model). A *model* is a morphism in $\mathbf{Para}(\mathcal{C})$ for a Markov category $\mathcal{C}$: a pair (P, f) of a *parameter object* $P \in \mathcal{C}$ and a kernel

$$f: P \otimes X \longrightarrow Y,$$

with X the *input object* and Y the *output object*. Composition of $(P, f): X \rightarrow Y$ with $(Q, g): Y \rightarrow Z$ is $(Q \otimes P, g \circ (\text{id}_Q \otimes f))$.

In part: the separation of P from X is carried by the formula IR, in which every input of a model version declares a role — **feature**, **parameter** or constant — and the input contract and the parameter inputs are computed from it (`maya/formula/ir.py: input_contract, parameter_inputs; tests/test_formula.py::test_pylift`). There is no general **Para(Stoch)** object: kernels are deterministic expression trees, a declared black-box node, or a vendor endpoint, and stochastic output is not represented.

In plain terms. A model is a machine with *dials*. X is what you feed in, Y is what comes out, and P is the dials that decide how the machine behaves. The definition insists on keeping the dials separate from the machinery, because that separation is where all the leverage is. “Kernel” means the output may be random: feed in the same input twice and you may get different answers. That covers a language model sampling text. Deterministic behaviour is the special case where the randomness is trivial. The composition rule says: plug one machine into another and the combined machine’s dials are *both* sets of dials together. Nothing is absorbed.

Definition 3.2 (Fitting morphism). A *fitting morphism* for (P, f) is a morphism $\varphi: D \rightarrow P$, where D is an object of *fitting evidence*. A model is *accompanied* when φ is available to the governing party and *opaque* otherwise.

Theorem 3.2 is where the account departs from received practice. Estimation, calibration, training, elicitation, configuration and authorship are not different *kinds of model*; they are different fitting morphisms into the parameter object of one kind of thing.

Worked example: two artefacts that look nothing alike

A **Black–Scholes pricer** takes spot, strike, tenor, rate and volatility and returns a price. Its formula comes from theory. There are no dials: P is terminal. Feed it the same inputs and it returns the same price forever.

A **SABR volatility surface** takes a strike and a tenor and returns an implied volatility — but first, four parameters must be solved against this morning’s quotes. Those four numbers are P , and they are different every day.

Same shape, entirely different operational reality, and the difference lives in P .

3.3 Trainability is derived

Proposition 3.3 (Classification). Let $\kappa(P)$ record whether P is terminal and whether it is accessible, let $\varphi \in \{\text{NONE}, \text{CALIBRATE}, \text{ESTIMATE}, \text{TRAIN}, \text{CONFIGURE}, \text{ELICIT}, \text{AUTHOR}\}$ name the fitting morphism, and let $\text{ad} \in \mathbb{B}$ record whether the artefact updates P in production. Then the informal taxonomy of governed artefact kinds is the value of a total function of $(\kappa, \varphi, \text{ad})$, computed in this order:

$$\begin{aligned} P \text{ inaccessible} &\mapsto T6 \\ P \cong I \text{ terminal} &\mapsto T0 \\ \varphi = \text{TRAIN} \wedge \text{ad} &\mapsto T4 \\ \text{otherwise, by } \varphi : &\quad \text{CALIBRATE} \mapsto T1, \quad \text{ESTIMATE} \mapsto T2, \quad \text{TRAIN} \mapsto T3, \\ &\quad \text{CONFIGURE} \mapsto T5, \quad \text{ELICIT} \mapsto T7, \quad \text{AUTHOR} \mapsto T8. \end{aligned}$$

The classification is never stored and never declared.

Not in MAYA: the nine-way classification. The rebuild records a model’s *kind* — `formula`, `black_box`, `composite` or `vendor` (`maya/services/models.py`) — and that kind is declared by whoever registers the model, which is D1 in the form this section argues against. Two rows survive in a weaker form. The T0 split is derived: a model whose IR declares no parameter input may be licensed to run without a training warrant, and one that declares any may not (`maya/services/execution.py`). T6 exists as the declared kind `vendor`, whose warrants behave as an internal model’s while the bundle states that it cannot re-execute and holdout scoring refuses by name (`tests/test_vendor_models.py`). The fitting morphism φ and the adaptivity flag are not recorded anywhere.

Proof. The function is total because the cases are checked in order and the final dispatch has a default. The two extremal cases are the mathematically interesting ones. T0 is $P \cong I$: since I is terminal, $I \otimes X \cong X$, the model is a bare kernel $X \rightarrow Y$, and the fitting problem is vacuous — so a demand for fitting evidence is not an unanswered question but an ill-typed one. T6 is the case in which P exists but neither P nor φ is available, so the only accessible datum is the composite $f(p, -)$ for the vendor’s fixed p , which is an ordinary morphism $X \rightarrow Y$; that is precisely why behavioural evidence is the only evidence obtainable. Exhaustiveness of the remaining rows over current practice is an empirical claim about the domain, not a mathematical one, and we make it as such. $\square$

Class	Parameter object P	Fitting morphism φ	Refresh trigger
T0 Analytical	terminal	none; nothing to fit	specification or regulation
T1 Calibrated	calibration set	a solver on quoted instruments	each market close
T2 Estimated	coefficients	an estimator on a sample	a periodic refit
T3 Learned	weights	a training run	schedule, drift, decay
T4 Adaptive	weights that move in production	a training run, then itself	continuously, by the model
T5 Configured	base model, prompt, corpus, tools	configuration and retrieval	any of those, or the base model
T6 Opaque	exists, unreachable	somebody else’s, unavailable	a vendor release
T7 Elicited	weights a panel agreed	an elicitation protocol	the committee cycle
T8 Authored	a rule set	authorship	a policy change

In plain terms. Practitioners argue endlessly about whether a pricer “is a model”, whether a rule set counts, whether a chatbot belongs in the inventory. The table says the argument is misconceived. These are not nine kinds of thing. They are one kind of thing, distinguished by *how the dials get set* — and because that is computed rather than typed in, nobody can get it wrong on a form.

The two ends are worth dwelling on. At the top the dials do not exist, so asking about them is meaningless. At the bottom of the opaque row the dials exist but are behind a contract, so asking about them is futile. Everything else is in between.

Corollary 3.4 (Fitting evidence is typed). *A demand for fitting evidence is well-typed exactly when the class is not T0 or T6. In the first case there is no P to inhabit; in the second there is no accessible φ to evidence.*

In part: for T0 only, and without a named test. An execution warrant is drawn directly on a model version with no parameter inputs, and its `parameters_approved` check passes with “no parameter set required”; a closed-form model with a parameter input is refused with “a trainable model needs a training warrant and an approved parameter set” (`maya/services/execution.py`). No committed test exercises either branch. Section 3.6 records where the check is permissive.

Worked example: three artefacts through the same lens

An SA-CCR exposure calculator. The formula is prescribed by regulation; P is terminal. There is no dataset, no estimator, no retraining schedule. What must be evidenced is that the implementation computes what the regulation says — a test suite against reference values, not a validation sample. Any system demanding a training set here asks a meaningless question and gets a meaningless answer typed into a mandatory field.

A small-business PD scorecard. P is forty-two coefficients; φ is a logistic regression over eight years of originations. Evidence: the sample, its representativeness, the estimator, out-of-time performance, calibration.

A bought-in bureau score. P exists — somebody at the bureau fitted it — but neither P nor φ is available. Demanding development documentation is futile. What you can do, and what the structure says is the *only* thing you can do, is watch the composite: compare its outputs against your own realised defaults. That is not a compromise forced by commercial reality; it is what the mathematics permits.

3.4 The classification is the index

Almost every structure in this domain is a family indexed by something else: versions by artefact, calibrations by version, deployments by environment, and — the one that matters here — *evidence requirements by kind of artefact*. The mathematics of indexed families is the fibration, and the governance question it answers is what a new kind of artefact costs.

Let $p: \mathcal{E} \rightarrow \mathcal{B}$ be a fibration with $\mathcal{B}$ a category of artefact kinds, and let the fibre $\mathcal{E}_c$ over kind c carry the four things that vary with kind: an evidence schema, a lifecycle, a set of monitors that can answer a question about it, and a set of documents that compile for it.

Proposition 3.5 (Conservative extension). *Let $\mathcal{B}^+$ extend $\mathcal{B}$ by a new object c with a chosen fibre $\mathcal{E}_c$ and no new morphisms into or out of existing objects. Then the inclusion $\mathcal{E} \hookrightarrow \mathcal{E}^+$ is full and faithful, commutes with the projections, and satisfies $\mathcal{E}_b^+ = \mathcal{E}_b$ for every $b \in \mathcal{B}$. No judgement expressible over $\mathcal{B}$ is altered by the extension.*

Proof. $\mathcal{E}^+$ is the coproduct $\mathcal{E} + \mathcal{E}_c$ over $\mathcal{B} + \{c\}$; fullness, faithfulness and fibrewise identity are immediate, and the cartesian liftings over $\mathcal{B}$ are unchanged since no morphisms were added. $\square$

Proposition 3.6 (Totality). *The extension claim is worth exactly the totality of the fibration: if some fibre is empty in one of the four facets, there is a kind about which the system has nothing to say, and it will say nothing by omission rather than by refusal. Totality is therefore a checkable property — no fibre is empty — and it is checked before the system serves rather than when a caller relies on it.*

Not in MAYA: there is no fibration. Evidence requirements vary by object type — feature, feature set, model, parameter set, warrant — through the workflow policy’s named checks, not by model kind, and nothing checks that every kind has a complete set of them. Totality is therefore a property of the argument here and not of the system.

In plain terms. Think of a filing cabinet where each drawer holds one kind of artefact and comes with its own forms: what must be filled in, what gets monitored, what documents come out. The bad design writes the list of drawers into the cabinet’s frame, so adding a drawer means rebuilding the cabinet. The good design makes drawers independent, so adding one is adding one — and Theorem 3.5 is the guarantee that adding one cannot disturb what is in the others.

Theorem 3.6 is the other half, and it is the half that is usually missing: every drawer has to actually contain its forms. A drawer with no forms in it is worse than an absent drawer, because the cabinet looks complete.

The base has to be derived

There is a constraint on $\mathcal{B}$ that the fibration literature has no reason to state and that this domain does. It is the paper’s own thesis applied to the paper’s own machinery, and it is sharp enough to be a dilemma.

Proposition 3.7 (A declared base admits no total fibration). *Suppose the base $\mathcal{B}$ is a free-text attribute recorded by a person. Then totality (Theorem 3.6) and conservative extension (Theorem 3.5) cannot both be delivered:*

1. *If the admissible values are closed, then extension requires enlarging the closed set, which is a change to the base rather than the supply of a fibre — and Theorem 3.5 no longer describes what adding a kind costs.*
2. *If they are open, then a value with no fibre is producible at any time by writing one down, so the totality check quantifies over a set that does not contain the values actually in use. The gate passes and the fibration is not total.*

Both horns are avoided exactly when the base is derived: an index computed from the artefact cannot be typed wrong, cannot be extended by accident, and cannot disagree with the artefact it indexes.

Proof. The two cases are exhaustive for an attribute whose values are supplied rather than computed. In the closed case, admitting a new kind changes $\mathcal{B}$ ’s object set by an act external to supplying $\mathcal{E}_c$, which is what Theorem 3.5 assumes is the whole of the extension. In the open case, let b be a value no fibre covers; the totality check ranges over the registered indices, which by construction excludes b , so it returns success on a fibration that has no fibre over b . If instead the index is a function of the artefact into a fixed finite set — as κ , φ and ad are in Theorem 3.3 — neither case arises: the index set is fixed, so totality quantifies over all of it, and extension is the supply of a fibre. $\square$

The trainability classification of Theorem 3.3 is such a base; the declared **kind** of MAYA is the closed case of the first horn, which is harmless only because nothing is indexed by it. That is the reason it is worth deriving twice over: once so that a closed-form pricer is a type rather than an exception, and once so that the structure indexed by it can be required to be complete. An organisational label — what desk owns this, what business it serves — remains useful for grouping and reporting, and indexes nothing.

Worked example: a monitor that ran for two years and meant nothing

A discount-curve pricer is registered and somebody attaches a *performance* monitor: discrimination against realised outcomes, evaluated monthly. It runs. It never fires. On the estate screen the model is green and monitored.

The pricer has no parameters and no fitted relationship to outcomes, so discrimination is not a question about it. The monitor was not failing to detect anything; there was nothing of that kind to detect. And this is worse than an absent monitor, because an absent monitor appears in the coverage worklist and a meaningless one does not — it reads as coverage.

With a fibre over the class, the monitor kind is checked against what the class can answer, and the attachment is refused with what *is* answerable for a T0 artefact named instead: whether the inputs are still inside the range it was benchmarked over. The refusal is not a restriction on what validators may do. It is the fibre being read rather than merely held.

3.5 When the parameter object can be inspected

For most classes P is a vector of numbers and the interesting questions about it are statistical. For T8 it is an authored rule set, and that changes what is askable: a rule set has *logical* structure, so questions about it can be decided rather than estimated.

The condition for this is a restriction, and the restriction is the point. If rules are written in a general expression language, questions about them are questions about programs. Confine a condition to comparisons on declared fields combined by AND, OR and NOT — no arithmetic, no function calls, no free text — and each condition has a disjunctive normal form whose conjunctions reduce to a domain per field: an interval, a permitted set, an excluded set and a null state. Emptiness and containment are then arithmetic.

Proposition 3.8 (Reachability under first-match-wins). *Let a rule set be an ordered list of rules with a required otherwise clause, evaluated first match wins. Under the restriction above, “rule r_j is shadowed by some single earlier rule r_i ” is decidable, by testing whether r_i ’s condition covers r_j ’s. The decision procedure is sound: a rule it reports as shadowed is unreachable. It is incomplete: a rule covered only by the union of two or more earlier rules is not reported.*

Not in MAYA: authored rule sets are not a model kind, and no reachability analysis exists. The nearest structure is the workflow policy editor, which refuses an unreachable *state* and a transition no approver could satisfy at edit time (specification §10.6; `maya/workflow/policy.py`) — reachability over a state machine, not over rules.

Proof. Soundness: if r_i ’s condition covers r_j ’s, then every input reaching r_j in evaluation order has already matched r_i , so r_j never fires. Incompleteness is exhibited by $r_1 = (x > c)$, $r_2 = (x \leq c)$ and any r_3 : no single earlier condition covers r_3 , and their union covers everything. $\square$

We state the limit exactly rather than describing the check as “unreachable rule detection”, because the gap is the interesting part. Complete coverage checking is satisfiability over the theory, decidable here but requiring a solver; the promise made is the one delivered — *no rule is shadowed by any single earlier rule* — and a check that claims more than it delivers is precisely the failure mode the analysis exists to find.

In plain terms. Rule sets are read top to bottom and the first match wins. That makes them easy to read and quietly easy to get wrong: a rule that an earlier rule already covers can never fire, and nothing about reading the document tells you so. Somebody believes that rule is in force. It appears in the model card, gets cited in a committee paper, and survives every review — because a rule that never fires also never produces a wrong answer.

The analysis finds those, and it is deliberately honest about its reach: it catches a rule blocked by one earlier rule, not a rule blocked by two of them working together. Saying which is the difference between a check people trust and a check people switch off.

Remark 3.9 (Why this belongs in this paper). Two of the account’s threads meet here. Theorem 3.3 says T8 is a class because of *how* P is inhabited — somebody wrote it down — and Theorem 3.8 says that when P is inhabited that way, and is held in a structured form rather than as text, properties of P become derivable facts about the model. An authored rule set is the most declared parameter object there is, and it is the one whose contents the platform can say the most about.

3.6 What the derivation still reads

Honesty requires the converse. Theorem 3.3 derives the class from $(\kappa, \varphi, \text{ad})$, and of those three, κ is a property of the artefact while φ and ad are recorded by a person. The derivation therefore *shrinks* the declared surface — from a nine-valued taxonomy with nine ways to be wrong, to two facts about the act that produced the parameters — rather than eliminating it. Two consequences deserve statement.

First, the shape of P does not decide the class; the act does. A kernel whose parameters are recorded as learned weights but whose fitting morphism is an estimator is T2, and correctly so: twenty coefficients from a solver are an estimation whatever the storage is called. Nothing

cross-checks the two declarations against each other, and a disagreement between them would not be a finding.

Second, the fallback direction is permissive. Where φ is unrecorded and P is non-terminal, a dispatch that defaults to T0 exempts the artefact from fitting evidence, and an artefact whose parameters exist but whose provenance nobody recorded is treated like a closed-form pricer. This is the same failure direction D2 names, one level in.

MAYA exhibits it, in the one row it derives. Its T0 test reads the parameter inputs only of a model whose IR has a closed-form body. A model without one — a declared black box, a vendor model, a composite — passes as “no parameter set required” whatever parameters it declares, and so does an execution warrant drawn from a training warrant without naming a parameter set: probed directly, both are created and approved with the `parameters_approved` check reporting *non-trainable model* for a model that declares a parameter (`maya/services/execution.py: create, check_params`). The training warrant itself is stricter — it will not seal without an approved parameter set — so the gap is at the second licence, not the first. We report it rather than repair it here; it is the defect this subsection predicts, found where the derivation stops reading structure and starts reading an absence.

3.7 Points of P , and what a version is

Proposition 3.10 (Refitting preserves the morphism). *Let (P, f) be a model and $p, p' \in P$ two inhabitants obtained by the same fitting morphism $\varphi: D \rightarrow P$ from data d, d' . Then (P, f) is unchanged: p and p' are points of the parameter object, not distinct morphisms. Consequently the pair $((P, f), p)$, not (P, f) alone, determines observable behaviour.*

Proof. Immediate: φ has codomain P , and f is a fixed morphism independent of which point of P is supplied. $\square$

Mathematically trivial; consequential in practice. Standard practice mints a new *version* on every refit, conflating a change of morphism (new φ , P , X or Y) with a change of point (same everything, new data). A supervisor asking whether a model changed between two dates then asks a question with two answers. Separating them makes an inhabitant of P a first-class record with its own identity, immutability and provenance — which φ produced it, from which d , over which window — so that a daily recalibration governs the *procedure* once while a periodic re-estimation governs *each* inhabitant, and both are the same construction at different frequencies.

Runs: a model version’s identity is the canonical hash of its formula IR, its code artefact and its input contract, and nothing else (`maya/services/models.py, _freeze`); the IR hash ignores the rendering and not the mathematics (`tests/test_formula.py::test_hash_ignores_latex_but_not_math`). Parameters are a separate versioned object: a `ParameterSet` is uploaded against a training warrant, checked against the checksum of the data that warrant issued, and approved on its own, and one model version may carry several (`maya/services/warrants.py, upload_parameters; tests/test_warrants.py::test_the_checksum_cycle_seal_score_execute_and_bundle`). A refit is therefore a new point of P and never a new version. What is not recorded is φ itself: the parameter set names the warrant and the data, not the act that produced the numbers.

Proposition 3.11 (Parameter accumulation). *For composable $(P, f): X \rightarrow Y$ and $(Q, g): Y \rightarrow Z$, the composite has parameter object $Q \otimes P$. In any finite network the composite parameter object is the tensor of the parameter objects of all constituents reachable from the inputs.*

Proof. Immediate from composition in **Para**($\mathcal{C}$) and induction on network size. $\square$

Runs: a composite model — ensemble, pipeline, router, residual or hierarchical — carries *one* parameter set, namespaced by member alias (`base.sigma, skew.beta`) with the combiner’s own parameters bare; evaluation hands each member exactly its own slice (`maya/formula/evaluate.py, evaluate_composite`; specification decision D-8; `tests/test_formula.py::test_composite_union_maturity_seeds_and_eval, ::test_composite_reference_code_matches_the_evaluator`). That is $Q \otimes P$ with the tensor written as a prefix. A

composite’s bundle re-executes the whole of it offline (`tests/test_sdk_modes.py::test_a_composite_model_is_re_executed_by_the_bundle_verifier`).

Corollary 3.12 (Change closure). *Any change to the inhabitant of a constituent parameter object changes the inhabitant of the composite parameter object. Recalibration of an upstream artefact is therefore, formally, a change event for every downstream artefact.*

In part: within a composite only, and as a notification rather than an event. A member’s parameters are part of the composite’s one parameter set, so a member refit is a new composite parameter set. Across models there is no such closure: MAYA does not record one model’s output feeding another outside a composite. Deprecating a model version notifies the owner of every warrant that depends on it (`maya/services/models.py, _warn_dependents`); the specification’s re-approval of a composite whose `tracking` member moves (§8.7) was not found in the code.

Worked example: the daily recalibration nobody calls a change

At 6:30 each morning a discount curve is re-solved against market instruments; its parameter set changes. Downstream sit a swaption pricer, an XVA engine, a VaR engine, a capital calculator and a capital plan. By Theorem 3.11 every one of them has had its parameter object change. Formally each has changed; operationally the change-management process records nothing, because curves are “market data”.

Both readings are defensible and must be reconciled explicitly rather than by silence. The usual resolution distinguishes routine recalibration *within a declared tolerance*, governed by monitoring, from recalibration outside it, which is a change event. The formalism does not choose; it forces you to notice that a choice is being made, and to write the tolerance down.

4 One order

4.1 Four questions that were one question

Four places in a model register ask whether one thing fits where another fitted.

Where	What it asks
An alias move	may this replacement version stand where the incumbent stood?
A fit warrant	does this featureset provide what the kernel declares it reads?
A contract comparison	does this operating contract refine the one it replaces?
A dependency edge	does what the source produces arrive where the target reads it?

In the registers we have examined each row has its own answer: a variance routine, a slot loop, a contract refinement check and — for the fourth — frequently nothing at all. Four implementations of one relation are four opportunities to disagree, and the disagreement is directional: toward permitting more, because a check that wrongly permits produces no bug report. The fourth row is the worst of them, because a dependency graph whose edges were never checked is followed by a blast radius that trusts them.

MAYA places the questions differently — it has no alias move and no operating-contract object — and answers them with three routines rather than one: a version-change classifier, a warrant’s contract check and a composite’s union contract. Theorem 4.9 gives the map, and the three do in fact disagree in one small place, recorded there. That is the prediction of this subsection holding in miniature, in a system written by somebody who had already made the argument.

4.2 The order

Let a *field* be a name, a datatype, a nullability flag and optional numeric bounds; a *schema* a finite set of fields with distinct names.

Definition 4.1 (Field acceptance). Field u *accepts* field v , written $u \succcurlyeq v$, when u and v have the same datatype; u is nullable if v is; u 's lower bound is no greater than v 's; and u 's upper bound is no less than v 's, where an absent bound is the widest.

Definition 4.2 (The schema order). For schemas A, B ,

$$A \sqsubseteq B \quad \text{iff} \quad \text{for every field } v \in B \text{ there is } u \in A \text{ with the same name and } u \succcurlyeq v.$$

Read $A \sqsubseteq B$ as A *can stand in for* B .

Not in MAYA: the order as a function. What runs is its restriction to names and datatypes, three times over (Theorem 4.9). Bounds are declared on parameter inputs only, and nullability is not compared anywhere.

A may carry fields B does not; they are simply not read. Each shared field must accept *at least* what B 's accepted, because a replacement that rejects an input its predecessor took is a replacement that breaks a caller. This is contravariance in inputs [9, 10], stated as an order rather than as a rule.

In plain terms. “Can stand in for” means: you provide everything the other one provided, and you are no fussier about any of it. Extra things you provide are fine — nobody has to look at them. Being fussier is not fine, because someone was relying on the old tolerance.

Remark 4.3 (The direction is counterintuitive, and reliably so). The order runs opposite to the intuition most readers bring to it. “Refinement” suggests narrowing, and a subtype is narrower — in the set of values it denotes. But a schema here is read as a set of *inputs a slot will accept*, and standing in for something requires accepting at least what it accepted. So a wider acceptance is admissible and a narrower one regresses, which is the reverse of the reading the word invites.

This is why the relation is worth writing as an order with a stated direction rather than as a rule of thumb in prose. The two readings of “narrower” are easy to hold at once and impossible to hold consistently, and the failure they produce is silent: an order stated in the intuitive direction admits exactly the replacements that break callers.

Proposition 4.4 (Preorder, and a partial order on canonical form). $\sqsubseteq$ *is reflexive and transitive. It is antisymmetric up to the ordering of fields: if $A \sqsubseteq B$ and $B \sqsubseteq A$ then A and B have the same fields as sets. On schemas presented in name-sorted canonical form — the form a system comparing them should construct — it is therefore a partial order.*

Not executed: no test asserts the order laws over generated schemas.

Proof. Reflexivity is immediate. For transitivity, $\succcurlyeq$ is transitive: datatypes chain by equality; nullability chains by implication; and for the lower bound, if u 's bound is absent the condition holds vacuously, and otherwise v 's is present and no greater, hence w 's is present and no greater — and symmetrically above. For antisymmetry, mutual comparability forces the same names, equal datatypes, equal nullability and, in each direction, an inequality on each bound, so the bounds coincide. $\square$

4.3 It is a lattice

Definition 4.5 (Meet and join). $A \sqcap B$ is the schema on the *union* of the field names, with each shared field widened: nullable if either is, the lesser lower bound and the greater upper bound, an absent bound absorbing. It is defined only when every shared name carries the same datatype in both. $A \sqcup B$ is the schema on the *intersection* of the names that agree on datatype, with each field narrowed: nullable only if both are, the greater lower bound and the lesser upper bound.

Theorem 4.6 (Schemas form a lattice). *On any finite set of field names, $\sqcap$ is the greatest lower bound and $\sqcup$ the least upper bound of $\sqsubseteq$, wherever $\sqcap$ is defined. The empty schema is the top element. There is no bottom.*

Not in MAYA: no join is computed anywhere, and no test asserts the lattice laws.

Proof. $A \sqcap B$ is a lower bound. Every field of A appears in $A \sqcap B$ with bounds no narrower, so $A \sqcap B \sqsubseteq A$, and symmetrically for B .

It is the greatest one. Let $C \sqsubseteq A$ and $C \sqsubseteq B$, and take a field n of $A \sqcap B$. If n occurs in only one of them, C accepts that field by hypothesis. If it occurs in both, C 's field at n accepts both, so its datatype agrees, it is nullable whenever either is, and its lower bound is either absent or no greater than each of theirs — hence no greater than their minimum, with the absent case handled by the definition of acceptance, which requires an absent bound on the right to be matched by an absent bound on the left. So $C \sqsubseteq A \sqcap B$.

$A \sqcup B$ is an upper bound. Each field of $A \sqcup B$ is the narrowing of a field present in both, so both accept it.

It is the least one. Let $A \sqsubseteq C$ and $B \sqsubseteq C$, and take $n \in C$. Both A and B carry n with the datatype of C 's field and with acceptance at least C 's, so n survives into $A \sqcup B$; its narrowed bounds are the tighter of two that each accept C 's, hence still accept C 's, and it is nullable whenever C 's is. So $A \sqcup B \sqsubseteq C$.

Top. The empty schema imposes no condition, so $A \sqsubseteq \emptyset$ for every A .

No bottom. A least element would have to carry every field name that could ever exist; over an unbounded name set no such schema is finite. The structure is a lattice on each finite fragment, which is the only fragment anything inhabits. $\square$

Proposition 4.7 (Order and meet agree). *$A \sqsubseteq B$ if and only if $A \sqcap B = A$ (up to the equivalence of Theorem 4.4), and $A \sqcap (A \sqcup B) = A$. Meet is idempotent, commutative and associative where defined.*

Not executed: mathematics only.

The link in Theorem 4.7 is what makes the structure a lattice *order* rather than an order with two unrelated binary operations [7]. We do not claim distributivity and do not need it.

In plain terms. Two schemas have a *meet*: the thing that can stand in for both. It has all the fields either of them had, each loosened enough to satisfy both. That is exactly the answer to “what would a single feature set have to provide in order to serve both these models?” They also have a *join*: what they agree on. Only the fields both have, each tightened to what both promised. That is the answer to “what may a consumer of either of these rely on?” And the empty schema is the top: it demands nothing, so everything can stand in for it.

4.4 The meet is partial, and informatively so

Proposition 4.8 (No meet). *Two schemas whose shared field name carries different datatypes have no lower bound, hence no meet.*

Runs: `maya/formula/composite.py::union_contract`, which raises `ContractMismatch` naming the attribute, both datatypes and the members that want each; `tests/test_formula.py::test_composite_union_maturity_seeds_and_eval`

Proof. A lower bound must carry that name with a field accepting both, and acceptance requires datatype equality, which cannot hold for two distinct datatypes. $\square$

The partiality is the useful part. An implementation should raise a named error carrying the conflicting slot and both datatypes, rather than returning an empty result — because the caller who asks for a meet has a decision to make, usually *these two models cannot share a featureset*, and an absent value threaded through three layers becomes a silent empty schema somewhere downstream.

In MAYA the meet is deployed and not law-tested, which is the opposite of the arrangement one would choose and the common one in practice: a composite model’s input contract is computed as the union of its members’ contracts, with aliasing where two members read one attribute under different names, and two members wanting one name at different datatypes is refused by name when the composite’s contract is computed (`maya/services/models.py, _contract`). That is the fourth line of Theorem 4.9, in use, on names and datatypes.

4.5 One relation, four answers

Proposition 4.9 (The four questions are one comparison). *Let $\text{in}(v)$ and $\text{out}(v)$ be the input and output schemas of a version, $S(F)$ the declared schema of a featureset, and $\text{in}(k)$ the input schema a kernel declares. Then*

$$\begin{aligned} \text{version } v' \text{ may replace } v &\iff \text{in}(v') \sqsubseteq \text{in}(v) \text{ and } \text{out}(v') \text{ provides } \text{out}(v), \\ \text{featureset } F \text{ serves kernel } k &\iff S(F) \sqsubseteq \text{in}(k), \\ \text{an edge } a \rightarrow b \text{ composes} &\iff \text{out}(a) \sqsubseteq \text{in}(b), \\ \text{one featureset serves } k_1, k_2 &\iff S(F) \sqsubseteq \text{in}(k_1) \sqcap \text{in}(k_2), \end{aligned}$$

where the second conjunct of the first line is the coarser covariant test on outputs: every name $\text{out}(v)$ promised is still present at the same datatype.

In part: three lines of four, each restricted to names and datatypes, by three routines. *Line one*, for features: a new version is classified **breaking** when the index changes or an attribute the previous version had is missing or retyped, **additive** when attributes are only added, and **behavioral** otherwise (`maya/services/catalog.py::change_class; tests/test_features.py::test_change_classes`) — exactly the covariant “provides” test. It classifies and does not refuse. *Line two*: a training warrant checks the model’s input contract against the feature set’s schema and lists every miss (`maya/services/warrants.py::validate_contract; tests/test_warrants.py::test_contract_mismatch_is_refused_listing_every_attribute`). *Line three* is not implemented: MAYA does not record one model’s output feeding another outside a composite, and inside a pipeline composite a member reads its predecessors’ outputs by name at evaluation time, with no definition-time check. *Line four* is the composite union contract above.

Three routines answering fragments of one relation is the arrangement the section argues against, and the prediction that they drift holds in miniature. The warrant check admits any numeric attribute — an integer, a decimal, a boolean — into an input declared `float64`, which is a widening; the union contract and the change classifier require the datatype to be identical. The widening is defensible for a numeric input; what is not defensible is that it is a decision nobody made in one place, and a test that read the three call sites for one function would have found it.

The fourth line is the meet doing work: a featureset that must serve two models must refine their meet, and when the meet does not exist, the honest answer is that no featureset can, with the offending slot named.

4.6 A refusal that names the remedy

Observation 4.10. The failure of $A \sqsubseteq B$ decomposes into two disjoint sets: names B has that A lacks (*missing*), and names A has that accept less than B ’s did (*narrowed*). They are different mistakes with different remedies: a missing slot means the wrong featureset was bound; a narrowed one means somebody tightened a constraint without noticing it was a promise.

In part: the warrant’s contract check reports the two failure kinds in different words — “needs attribute a , which the feature set does not expose” and “is t but a is t ” — and reports every one rather than the first (`maya/services/warrants.py::validate_contract`). Narrowing of bounds or nullability cannot be reported, because neither is compared.

This is a small thing that changes the character of a control. A refusal that says “incompatible” is a message; a refusal that says “does not provide `turnover`; accepts less than before at `dscr`” is an instruction.

4.7 Edits, and why independence matters

Schemas are not only compared; they are built, by inheriting from parents and applying edits. The edit operations — `ADD`, `DROP`, `OVERRIDE` — generate a monoid acting on schemas under merge-with-rightmost-wins, whose identity is the empty composition.

Proposition 4.11 (Edit laws). *Merge-with-rightmost-wins is associative with the empty map as two-sided identity, and is not commutative. Over this action: $DROP(x) \cdot ADD(x, \tau) = \text{id}$ where x is absent; $OVERRIDE(x, \sigma) \cdot OVERRIDE(x, \tau) = OVERRIDE(x, \sigma)$ with the later winning outright; and two edits naming different slots commute.*

In part: the action, not the laws. A feature or feature set may **extend** one parent; the child stores only its override diff, applied over the parent’s effective definition at every read, so a fix to the parent reaches every child (`maya/services/catalog.py: effective_feature_definition, apply_override`; `tests/test_features.py::test_derived_union_and_inheritance`). An **extend** that projects its parent down to the same attributes as a directly written set hashes the same, and the second is refused as a near-copy (`tests/test_featureset_proofs.py::test_project_of_extend_is_recognised_as_the_directly_written_set`). No test states associativity, identity or the commutation of independent edits.

Non-commutativity of the merge is required rather than incidental: a commutative merge has no notion of override, so a child could not correct what it inherits. Commutation of *independent* edits is the property two people editing a shared featureset rely on without knowing it — it is what makes the order in which their edits happened to arrive carry no meaning. It is stated only for independent edits, and deliberately: `OVERRIDE` then `DROP` of one slot is not `DROP` then `OVERRIDE`, and the second order is refused.

5 Composition, typed — and the risk that does not compose

5.1 An edge is a claim about types

Para($\mathcal{C}$) is symmetric monoidal, so a network of models is a string diagram built from generators by sequential composition, parallel composition, copying and discarding. Type-correctness of a wiring is decidable. In a register this becomes a check on the relation that records one model’s output being read by another.

Proposition 5.1 (Composition type-check). *An edge $a \rightarrow b$ asserting that a ’s output is read as an input by b holds only if $\text{out}(a) \sqsubseteq \text{in}(b)$. Where it holds, the composite $b \circ a$ has the derived signature $(\text{in}(a), \text{out}(b))$.*

Not in MAYA: an edge check. MAYA records no model-to-model wire outside a composite, and a pipeline composite’s members read their predecessors’ outputs by name at evaluation time (`maya/formula/evaluate.py::evaluate_composite`); a name the next member reads and the previous one does not produce fails when the composite is evaluated, not when it is defined.

Three consequences follow from taking the check seriously.

Extra outputs are fine and a missing one is not. By Theorem 4.2, an output schema richer than the target reads composes — the surplus is simply unread — while a name the target reads and the source does not produce is a wire to nowhere, and the refusal names it.

The composite’s schema is derived, not declared. A composite whose signature somebody wrote down is a composite that can disagree with its parts. This is the same move as Theorem 3.3, one level out. MAYA does derive the composite’s *input* contract, as the partial meet of Theorem 4.8; its output is whatever the combine expression names.

And a composite may be governed as one unit. There are two defensible positions here and the choice between them is instructive. One authorises a *chain* of separately approved models in a single call, returns each node with its own descriptor, and deliberately signs nothing for the whole, on the ground that three approvals were given for three models and none for the composition. MAYA takes the other: it makes the composition itself the governed object. A composite — ensemble, pipeline, router, residual or hierarchical — is a model version with its own specification document, its own approval, one parameter set and one warrant over one feature set (specification §8.7, §9.5; `maya/formula/composite.py`). The objection to the first position is then met rather than avoided: there *is* an approval for the composition, because the composition is what was submitted. What it costs is that a composite is a new object to approve even when every member already is.

Two details carry the theory into that design. A composite’s maturity is capped at its *lowest* member’s — the meet on a finite chain of maturities, which composes, and which is precisely what Theorem 5.6 permits and no more; what does not compose is a risk *magnitude*, and none is produced. And every member below the target maturity is named, not the first (`maya/formula/composite.py: capped_maturity, blocking_members; tests/test_formula.py::test_composite_union_maturity_seeds_and_eval`). A refusal revealing one obstacle at a time is a refusal whose author has not computed the obstruction set.

Not every relation is composition. A register records several ways one object may stand to another, and only some of them describe a wire. MAYA’s lineage edges are typed — `extends`, `operand_of`, `derived_from`, `member_of`, `pinned_as`, `composite_member`, `trained_on`, `parameterized_by`, `executed_under` — and written in the transaction that creates the object (`maya/services/`). Only `composite_member` is a wire between models, and it is the only one whose types are checked. “Challenger of” or “benchmark for” would record how somebody thinks about a model; there is no wire, so there is nothing to type, and treating a challenger as a dependency would inflate every blast radius it appears in. The rebuild records neither, which is honest and incomplete.

Remark 5.2 (Vocabulary is part of the formalism). An early name for the model-to-model relation was *feeds*. In a bank a feed is market data, a reference file, a nightly drop, so “*A feeds B*” reads as though the governance platform consumed or produced one. It does neither: the edge is a statement about two entries in a register, describing a wire that somebody else’s engine carries. The name was wrong for what it asserts and was changed, and we record the episode because a formal account whose terms are read wrongly by its audience has a defect, and the defect is in the account rather than in the audience. MAYA’s word for its licence to compute — *warrant* — was chosen with the same care: a permission with a holder, a scope and an expiry, not a record of what happened.

Remark 5.3 (A variance choice, recorded as a choice). Theorem 5.1 applies $\sqsubseteq$ — the *input* order — to a pair whose left argument is an output. Presence and datatype are exactly right for composition. The bound and nullability conditions are the input rule applied one level out: they require the producer’s declared range to *contain* the consumer’s, so a producer with a strictly tighter range is refused. The alternative reading, that producing less is always safe, is the coarser covariant test of Theorem 4.9. A system holding both variance rules for outputs must say which applies where. We take the stricter one for composition, on the reading that a consumer fitted over a declared range and then fed a narrower one has had its input distribution changed without a change event — but that is a decision and not a derivation, and a reader is entitled to the opposite one provided it is made once and written down. MAYA compares no

bounds, so the question does not yet arise in it.

Worked example: the wire that was a drawing

An ECL stack records that a PD model feeds a provisioning engine. Both entries have existed for two years; the edge is on the diagram in every committee pack.

The PD model’s current version produces `pd_12m`. The provisioning engine’s current version reads `pd_lifetime`. Nobody wired the two systems — an adapter in between computes one from the other using a term structure that belongs to a third model, which is not in the register.

Under an unchecked edge the register reports a two-node dependency and a blast radius that is wrong in both directions: it claims a change to the PD model reaches provisioning directly, and it omits the term structure entirely. Under Theorem 5.1 the edge is refused at the moment somebody records it, with `pd_lifetime` named as the output that does not exist — and the conversation that follows is the one that discovers the third model.

5.2 Aggregate risk

Supervisory guidance asks that risk be assessed “both individually and in aggregate”, where aggregate risk “reflects interactions and dependencies among models; reliance on common assumptions, data, or methodologies” [40]. This expectation has precise formal content: it asserts the failure of compositionality.

Definition 5.4 (Risk assignment). Let R be a join-semilattice. A *risk assignment* is a function ρ from morphisms of $\mathbf{Para}(\mathcal{C})$ to R . It is *compositional* if $\rho(g \circ f) = \rho(g) \sqcup \rho(f)$ and $\rho(f \otimes h) = \rho(f) \sqcup \rho(h)$ for all composable pairs.

Compositionality is the assumption made implicitly whenever an aggregate is computed as the maximum or join of constituent ratings, which is standard practice.

Definition 5.5 (Fragility). Fix a network N with generators $G(N)$ and output ports $\text{out}(N)$. For $S \subseteq G(N)$ let $\text{corr}(N, S)$ be the output ports reachable in the diagram from some generator in S . The *fragility* of N is $\text{frag}(N) = \max_{g \in G(N)} |\text{corr}(N, \{g\})|$. A risk assignment is *fragility-sensitive* if $\text{frag}(N) > \text{frag}(N')$ implies $\rho(N) \sqsupset \rho(N')$ whenever the multisets of constituent risks coincide.

Theorem 5.6 (No compositional risk assignment). *Let R be a join-semilattice with at least two elements. No risk assignment is both compositional and fragility-sensitive.*

Proof. Let $a \sqsubset b$ in R . Take generators $A, A': X \rightarrow Y$ with $\rho(A) = \rho(A') = a$, and $B, C: Y \rightarrow Z$ with $\rho(B) = \rho(C) = b$. Let $N_1 = (B \otimes C) \circ \text{copy}_Y \circ A$, in which A ’s output is copied to both consumers, and $N_2 = (B \circ A) \otimes (C \circ A')$; both have two output ports. The multisets of constituent risks coincide, both being $\{\{a, b, b\}\}$. But $\text{corr}(N_1, \{A\}) = \text{out}(N_1)$ so $\text{frag}(N_1) = 2$, while every generator of N_2 reaches exactly one port so $\text{frag}(N_2) = 1$. If ρ is compositional then $\rho(N_1) = a \sqcup b = \rho(N_2)$, contradicting fragility-sensitivity. $\square$

Not in MAYA: any aggregate risk assignment. MAYA tiers each model on its own (Section 9) and computes no aggregate over models, and so neither confirms nor contradicts the theorem. It respects it in the only place it composes an assessment at all — a composite’s maturity, the meet over its members’ on a finite chain, which is compositional and makes no claim to be fragility-sensitive. The interaction premium of Theorem 5.8 is computed nowhere.

In plain terms. Two banks each run one curve model and two pricers. Bank A uses a single curve to feed both. Bank B built two curves independently, one for each.

Rate the models individually and you get identical answers: same curve rating, same two pricer ratings. So any method computing the total from the individual ratings gives both banks the same answer.

But Bank A has a single point of failure and Bank B does not. That is the entire theorem: *the risk of the whole is not a function of the risks of the parts, because part of it lives in the wiring.*

Remark 5.7 (The obstruction is the copy map). N_1 and N_2 differ in exactly one respect: N_1 uses copy_Y . Shared dependency *is* the comonoid copy map. In a Cartesian category copying is implicit and structurally invisible, so the distinction cannot be expressed at the level of the algebra — a concrete reason to prefer a Markov category as the ambient setting even when the artefacts under management are deterministic.

Corollary 5.8 (Laxity is necessary). *Any fragility-sensitive risk assignment is at best lax monoidal, $\rho(g \circ f) \supseteq \rho(g) \sqcup \rho(f)$, with the discrepancy attributable to shared constituents. We call $\rho(N) \ominus \bigsqcup_{g \in G(N)} \rho(g)$ the interaction premium.*

The practical reading is that a governance system should report the join of constituent risks *and* the composite, and attribute the difference: to common parameter objects (Theorem 3.11), common inputs, common fitting evidence, common methodology. The first three are computable from a typed graph; the fourth is not, and we do not pretend otherwise. MAYA’s typed lineage graph holds what the first two need — a model version shared by several composites, a feature pin shared by several feature-set pins — and answers the impact question over it; it computes no premium from it.

6 One operator

6.1 Two clocks

A fact carries two independent time coordinates: the *event time* at which it held in the world, and the *ingest time* at which the recording system learned it [16, 17]. Conflating them produces look-ahead leakage, which is the dominant silent failure mode in the domain [18]: an artefact fitted on facts that had not been recorded when the decision was taken assesses well and performs badly, and the discrepancy is attributed to degradation rather than to leakage.

The condition itself is folklore. What is not usually stated is that the *read* is an operator with laws, and that the laws are where reproducibility lives.

6.2 The read as an operator

Definition 6.1 (Point-in-time read). Let R be a finite set of records, each carrying event and ingest times in a totally ordered set. For a label time ℓ and an observation time a put

$$\text{Adm}(R, \ell, a) = \{ r \in R : r.\text{event} \leq \ell \wedge r.\text{ingest} \leq \min(\ell, a) \},$$

$$\text{AsOf}(R, \ell, a) = \arg \max_{r \in \text{Adm}(R, \ell, a)} (r.\text{event}, r.\text{ingest})$$

under the lexicographic order, undefined when $\text{Adm}(R, \ell, a) = \emptyset$.

Not in MAYA: the operator as defined. MAYA reads with a scalar knowledge bound and checks the per-row bound afterwards; Section 6.3 states exactly what that delivers and where it differs.

Both bounds are present because they refuse different things. The event bound is what the world had done by the moment of the decision. The ingest bound is what could have been *known* at that moment — and it is bounded by $\min(\ell, a)$ rather than by a alone, because a is a single scalar for a whole assembly and is usually much later than any individual label.

Proposition 6.2 (Four properties).

1. Idempotent. *If $r = \text{AsOf}(R, \ell, a)$ is defined then $\text{AsOf}(\{r\}, \ell, a) = r$.*

2. Commutes with projection. For any projection π that deletes non-clock columns, $\text{AsOf}(\pi R, \ell, a) = \pi \text{AsOf}(R, \ell, a)$.
3. Monotone in a . If $a \leq a'$ then $\text{Adm}(R, \ell, a) \subseteq \text{Adm}(R, \ell, a')$.
4. Saturating at ℓ . For all $a, a' \geq \ell$, $\text{Adm}(R, \ell, a) = \text{Adm}(R, \ell, a')$ and hence $\text{AsOf}(R, \ell, a) = \text{AsOf}(R, \ell, \ell)$.

Not executed: mathematics only: no property test asserts the four.

Proof. (1) r is admissible, so it lies in the admissible set of the singleton, of which it is the maximum. (2) Admissibility and the ordering key are functions of the two clocks alone, which π preserves; ties are broken by position, which π also preserves. (3) $\min(\ell, a) \leq \min(\ell, a')$, and the event bound does not involve a . (4) $\min(\ell, a) = \ell = \min(\ell, a')$, so the two admissible sets are the same set, and the arg max over it is the same record. $\square$

Remark 6.3. Property (3) is monotonicity of the admissible *set*, not of the chosen record: a later observation time may cause a different row to be selected. It says only that nothing which was knowable stops being knowable. Adm is likewise monotone in ℓ , by the same argument.

Corollary 6.4 (Saturation is the reproducibility guarantee). *A row assembled at any observation time at or after its label is the row that would be assembled at the label, and at every later observation time, whatever arrives in between. Reproducibility of a training set is therefore a property of the operator rather than a property of the discipline of whoever re-runs it.*

Not in MAYA: saturation as a property of the read. MAYA obtains reproducibility another way — a pin records its `as_of_known` and is sealed by the hash of its output — and recovers a weaker, checked form of saturation from its certificate (Theorem 6.8).

In plain terms. Every fact has two dates: when it was true, and when you found out. Ignore the second and you build training sets containing information that did not exist when the decision was made. The model looks excellent in testing — it has been shown the answers — and fails in production, and the post-mortem blames drift.

The rule is: use the latest fact that was both true by the decision date *and known by the decision date*. The second half is the one people drop, usually by bounding it with “now” instead.

And the reason it is worth stating as an operator rather than as a rule is the last property. Because the knowledge bound is the *earlier* of the decision date and the assembly date, re-running the assembly later cannot change the answer. Without that, a re-run quietly improves on the original — which is the least useful kind of reproducibility, because the numbers then agree with nothing, including themselves.

Worked example: the filed accounts that were restated

A borrower files Q1 accounts dated 31 March. They land in the warehouse on 20 May. The bank declines an application on 15 June. In August the borrower restates those accounts downward, from a debt-service ratio of 1.20 to 0.40.

Both rows are true and both belong in the store: same event time, different ingest times.

Read with one clock, the training row for the June decision gets 0.40, because that is the current value for Q1. The model learns to predict defaults using a number that exists *because* the default already happened.

Read with two, it gets 1.20 — and by Theorem 6.4 it still gets 1.20 when the set is rebuilt in 2028, after two more restatements. And the row labelled in September correctly gets 0.40, because by then the restatement was knowable.

6.3 A cut and a certificate, and what the pair is worth

MAYA does not implement Theorem 6.1. It implements two simpler things and relates them by a check, and the relation is worth stating exactly, because it is weaker than the operator in one respect and stronger in another.

A feature’s rows are keyed by an index whose first column is the event date, so each key k fixes an event time e_k and the records R_k for that key differ only in ingest time: a restatement is a new record under the same key, never an overwrite (`tests/test_properties.py::test_a_restatement_never_overwrites`). A resolution names one observation time a , `as_of_known`, for the whole read.

Definition 6.5 (The cut). $\text{Cut}(R_k, a) = \arg \max_{r \in R_k, r.\text{ingest} \leq a} r.\text{ingest}$, with a record of unknown ingest time admitted at every a and losing to any record whose time is known.

Runs: `maya/resolution/resolver.py::bitemporal_cut`; `tests/test_features.py::test_sc11_point_in_time_after_restatement`

Definition 6.6 (The certificate). For a declared lag λ (one day by default), put ℓ_k for the last instant of the day $e_k + \lambda$. A training warrant’s *leakage certificate* examines every resolved row whose ingest time is known, and records a violation for each row with ingest time after ℓ_k . It is issued *certified* when there are none, *certified with exceptions* when every violation carries a written justification and every non-causal fill is both allowed and justified, and *refused* otherwise; it is signed, and a warrant whose certificate is refused cannot be submitted.

Runs: `maya/services/warrants.py::leakage_certificate, check_leakage`; `tests/test_warrants.py::test_leakage_certificate_refuses_late_knowledge`

Proposition 6.7 (A clean certificate certifies the operator). *If the certificate records no violation at key k , then $\text{Cut}(R_k, a) = \text{AsOf}(R_k, \ell_k, a)$, up to ties in ingest time.*

Proof. Write $s = \text{Cut}(R_k, a)$. Every record of R_k has event time $e_k \leq \ell_k$, so $\text{Adm}(R_k, \ell_k, a) = \{r \in R_k : r.\text{ingest} \leq \min(\ell_k, a)\}$, a subset of the records the cut ranges over. No violation means $s.\text{ingest} \leq \ell_k$, and $s.\text{ingest} \leq a$ by construction, so $s \in \text{Adm}(R_k, \ell_k, a)$. Being the latest record in a superset, s is the latest in the subset; with the event time fixed, the lexicographic $\arg \max$ of Theorem 6.1 is the latest ingest time. $\square$

Corollary 6.8 (Reproducible or refused). *Let the certificate be clean at key k for a read at $a \geq \ell_k$. For every later read at $a' \geq a$, either the cut returns the same record, or the certificate records a violation at k .*

Proof. The cut at a' ranges over a superset. If its maximum lies in the old range it is the old record, by maximality. Otherwise its ingest time exceeds $a \geq \ell_k$, which is a violation. $\square$

In plain terms. The operator of Theorem 6.1 *chooses* the right row. MAYA chooses the latest row it knew about by one date for the whole read, and then *checks*, row by row, that nothing it chose was learned after that row’s own decision date. When the check is clean, the two agree (Theorem 6.7). And a later rebuild of a set whose decision dates had all passed cannot quietly change: it gives the same rows, or it fails the check (Theorem 6.8). That is saturation turned from a property of the read into a property of a signed document.

The difference is real and runs both ways. The operator never admits a late fact; the cut admits it and the certificate names it, so a late fact can be *accepted* — with a justification written on the warrant — which the operator cannot express and a validator sometimes needs. Against that, the certificate sees only what carries a clock, and in MAYA two kinds of row do not. A row produced by a fill has a null knowledge time, although its value was

learned when the row it was copied from was. And six of the feature algebra’s operators — projection, composition, coalescing, aggregation, resampling and case selection — return frames without the knowledge-time column, so a feature derived through any of them has no clock at all, and the certificate examines none of its rows (`maya/resolution/algebra.py`; `maya/resolution/resolver.py::apply_rules`). Both were found by writing this section and are reported rather than repaired here. Theorem 7.3 is the statement they violate.

6.4 A corollary about where the data may live

Theorem 6.4 is usually read as a statement about how to *query* a store. It is also a statement about which stores may be queried at all, and the second reading is the operational one.

The corollary holds because the knowledge bound is $\min(\ell, a)$ over rows that carry both clocks and are never amended in place: a restatement is a new row with a later ingest time, so the earlier read remains derivable. A source that overwrites — an ordinary warehouse table, a view maintained by a nightly job, a file replaced on a schedule — carries neither property. Its 20 May row is simply gone once the August restatement lands, and no operator applied at read time can recover what the June decision saw. The read does not fail; it returns 0.40 and says nothing, which is the failure mode Section 6 exists to eliminate.

So a platform that binds a featureset slot directly to an external table has not implemented Theorem 6.2; it has implemented a query that usually agrees with it. The two are indistinguishable until the first restatement, and then indistinguishable again afterwards, because nothing records that they diverged.

The consequence for a build is one line: *copy, do not connect*. An external source is pulled — read once, bitemporalised, and written into storage the platform controls as an immutable, versioned snapshot — and models read the snapshot. Connecting is cheaper and gives up the corollary. This is not a preference for one storage technology. It is the observation that Theorem 6.2 constrains its argument as much as its definition: an operator defined over rows that can be overwritten is defined over the wrong object, and the guarantee it appears to provide is a guarantee about a table that no longer exists.

Runs: `maya/services/sources.py` (a `sql` or `python` source is *pulled* into the feature’s bitemporal ingest log with a knowledge time -- the source’s own publication column when the definition names one, else the moment of the pull -- and is never read through at resolution); `tests/test_sql_source.py::test_pull_snapshots_the_query_and_restatements_append`; `tests/test_python_source.py::test_a_changed_result_is_a_restatement_and_the_past_stays_answerable`

In part: one source type connects. A `delta` source — another team’s Delta table — is read at resolution time and every row is stamped with the moment of the read as its knowledge time (`maya/services/feature_data.py::raw_frame`); no test exercises it. The failure is loud rather than silent — a read as of any earlier instant sees none of those rows, and every row older than the declared lag fails the certificate — but the rows’ knowledge time is the time of the read, not of the fact, which is precisely what this subsection says a connected source cannot supply.

6.5 Two properties the operator has to carry beyond itself

Theorem 6.2 is a statement about a function. A governance platform does not execute the training assembly — an engine somewhere else does — and it checks the assembly with a second computation of its own. So the operator has to hold in more places than the one it is defined in, and each of those is a property worth stating on its own.

The published rule is the applied rule. Where the point-in-time predicate is published as a string — in a plan, on a warrant — for engines the platform does not run, that string is a *contract*, and the property required of it is agreement: for every set of records, the rows

the published predicate admits are the rows the operator admits. The way to check that is to *execute* the published string against the same records and compare. Comparing the text instead passes on a rule that reads plausibly and admits different rows. MAYA’s certificate publishes its rule in words (“every row’s knowledge time $\leq$ its event date + λ day(s)”) and the same function that writes the words applies the rule, which is agreement by construction rather than by test.

The independent recomputation bounds the clock identically. Assembly verified by a second computation that deliberately does not reuse the assembly path is worth its cost only if both routes implement the *same* operator: a verifier bounding the ingest clock by the observation time while the assembly bounds it by $\min(\ell, a)$ disagrees on every set assembled after its labels matured, and therefore reports a mismatch on a correct assembly. A control that reports a violation where there is none teaches whoever reads the report to discount it. MAYA’s independent route is the reproducibility bundle: a `verify.py` shipped inside it recomputes every hash and re-executes the model on a machine with no MAYA (`maya/services/bundle.py`; `tests/test_sc2_aged_warrant.py::test_sc2_a_two_year_old_run_is_reproduced_from_its_warrant_alone`, which ages a whole training run by two years and reproduces its data and output hashes byte for byte from the bundle alone). It recomputes the bytes and not the certificate, so for this operator there is one route, not two.

A change of storage is not licence for a second implementation. Theorem 6.1 is defined over records and says nothing about the medium holding them. A platform that gains a second storage backend acquires, at that moment, the opportunity to write the predicate a second time. MAYA has two Delta Lake backends — the native `deltalake` bindings and its own pure-Python implementation of the protocol — and the cut is written once, above both, over the frames either returns (`maya/resolution/resolver.py`); what is written twice is the storage, and that is held to byte-identical results by one conformance suite run on each backend and by each backend reading the other’s tables (`maya_delta/`; `tests/test_maya_delta.py: test_conformance, test_cross_backend_round_trip`). The general form is the moral of Section 4: the number of implementations of a relation is a design variable, and every value above one is a standing decision to permit disagreement — which is why, where a second implementation is wanted for speed, one of the two is named the authority and the other is held to it. The column-at-a-time canonical encoder is an accelerator for the reference one, compared with it on randomised tables of every covered type (`maya/core/canonical_fast.py`; `tests/test_canonical_fast.py`); the vectorised fill rules are compared with the per-group rules on 96 randomised cases (`maya/resolution/grouped.py`; `tests/test_resolution_grouped.py`).

6.6 The input object, made an object

The operator reads records; something has to say which records. Treating the input object X with the same seriousness as P gives it a name and a version.

A *featureset* declares a schema of named, typed slots — and that schema, by Theorem 4.9, is what a kernel is defined over. A *version* of the featureset *fills* the schema, binding each slot to a specific feature and to the pinned version of the view supplying its values. The separation does three things. Swapping which feature fills a slot does not change the model’s input space; it changes what the model was fitted on, which is a different event with a different control. A version that cannot fill the schema is refused, because adding a slot is a change to X and therefore a model change rather than a data change. And because every binding pins a version, one featureset version resolves to the same bytes on every read — a stable identifier over moving contents being the failure this design exists to prevent.

In part: MAYA’s version/pin split is this separation with the roles relabelled: a *version* freezes how a feature set is assembled, a *pin* freezes what it assembled, and a feature-set pin refuses to exist until every member is

pinned, with a cascade that pins all members in one transaction or none (`maya/services/featuresets.py`; `tests/test_warrants.py::test_featureset_refuses_to_pin_unpinned_members_without_cascade`, `tests/test_featureset_proofs.py::test_a_forty_member_cascade_rolls_back_whole_then_seals_whole`). A pin’s identity is the hash of a canonical encoding of its rows — independent of column order and arrival order, and of the engine that wrote it — so the same data pinned twice hashes the same and changed data does not (`maya/core/canonical.py`; `tests/test_features.py::test_sc1_repin_is_byte_identical_and_changed_data_is_not`; `tests/test_properties.py`). A feature-set *version*, unlike the paper’s, may name members by version rather than pin, and then resolves to whatever those versions resolve to today; only the pin is stable.

7 One polynomial

7.1 Evidence is a derivation, not a log

Assurance evidence forms a directed acyclic derivation: fitting evidence supports a fitting run, which with code and environment supports a version, which supports test outcomes, which support a validation conclusion, which supports an authorisation, which supports a deployment. Governance claims are derived from base evidence by conjunction (joint dependence) and disjunction (alternative routes).

Annotate each base item with an element of a commutative semiring $K = (K, \oplus, \otimes, 0, 1)$ and propagate with $\otimes$ for joint dependence and $\oplus$ for alternatives. This is exactly the setting of provenance semirings [11].

Proposition 7.1 (Universality). *Let $\mathbb{N}[X]$ be the semiring of polynomials with natural coefficients over evidence identifiers X . For every commutative semiring K and valuation $\nu: X \rightarrow K$ there is a unique semiring homomorphism $h_\nu: \mathbb{N}[X] \rightarrow K$ extending ν , and for every derivation d ,*

$$\text{eval}_K(d, \nu) = h_\nu(\text{eval}_{\mathbb{N}[X]}(d)).$$

Not in MAYA: any evidence derivation. MAYA records evidence as typed lineage edges, a hash-chained audit log, custody events on warrants and signed certificates; it does not annotate them with a semiring or evaluate a derivation. What does run is three valuations over the *support* of a derivation, taken up in Section 7.4 — and the section is worth reading for what they get wrong, which is the subject of Theorem 7.5.

Proof. $\mathbb{N}[X]$ is the free commutative semiring on X [11]; the displayed equation is the universal property applied to the evaluation, which uses only $\oplus$ and $\otimes$. $\square$

The content of Theorem 7.1 for a governance system is that the same traversal answers a different question for each semiring, and that this is a theorem rather than a coincidence: where a direct evaluation and a pushforward disagree, one of the two routes is not a homomorphism, which is a fact about the structure and not about the traversal.

Semiring	$\oplus, \otimes$	Governance question answered
$\mathbb{B}$	$\vee, \wedge$	Is the claim supported at all?
$\mathbb{N}$	$+, \times$	How many independent derivations corroborate it?
$\text{PosBool}(X)$	absorptive $\cup$, pairwise $\cup$	Which minimal evidence sets suffice?
$\mathbb{N}[X]$	$+, \times$	Exactly how was it derived?
$([0, 1], \max, \times)$	$\max, \times$	With what confidence?
$(\mathbb{R}^+ \cup \{\infty\}, \min, +)$	$\min, +$	At what least cost can a gap be closed?
$(2^{\text{Regimes}}, \cap, \cup)$	$\cap, \cup$	For which regimes is the evidence admissible?

Remark 7.2 (Which semiring the minimal-support one is). The minimal-support semiring one reaches for in practice applies absorption — a set of supports is reduced to its minimal members, so $a \oplus ab = a$. With absorption this is the semiring of positive Boolean expressions $\text{PosBool}(X)$ rather than the $\text{Why}(X)$ of [13], which is idempotent but not absorptive; the two

sit at different levels of the provenance hierarchy [11, 14]. The distinction matters for what is retained: PosBool answers “what is the smallest set I could show an examiner”, and discards the fact that a claim had a second, larger, independent route. $\mathbb{N}[X]$ retains both — coefficients count distinct derivations and exponents count repeated use of one fact — which is precisely the information Boolean provenance throws away, and the reason to evaluate in the free object once rather than in each semiring separately.

In plain terms. A semiring is just “a way of doing arithmetic”: something playing the role of plus, something playing the role of times.

Tracking evidence has a fixed shape — this rests on that test AND that approval; this requirement can be met by this route OR that one. Keep the shape and swap the arithmetic and you get a different answer to a different question from the same computation. True/false arithmetic tells you whether the claim holds; probability tells you how much to trust it; least-cost tells you the cheapest way to close a gap; sets-of-evidence tells you what to put in front of an examiner.

The polynomial is the version that keeps everything, so all the others can be read off it afterwards rather than recomputed.

7.2 One layer across: derived features

A derived feature is a term: $Z = f(X, Y)$, computed from other features by a small expression language. The same construction applies, and gives a data-engineering rule the status of a theorem.

Proposition 7.3 (The ingest clock is a homomorphism). *Let $\text{prov}(Z) \in \mathbb{N}[X]$ be the evaluation of Z ’s derivation with each base feature sent to its own variable, and let ν send each base feature to its ingest time. Then the time at which Z became knowable is $h_\nu(\text{prov}(Z))$ computed in the semiring $K_\perp$ of Theorem 7.6, whose $\oplus$ and $\otimes$ are both $\max$ on times; equivalently, for $\text{prov}(Z) \neq 0$, the maximum ingest time over the free variables of $\text{prov}(Z)$.*

In part: for feature sets, not for the feature algebra. A feature-set row’s knowledge time is the latest of its members’ knowledge times on that row — the $\otimes$ of joint dependence (`maya/resolution/featureset.py`). Among the feature algebra’s operators, those that pass rows through — union, intersection, difference, rename, transform, lag — keep each row’s own clock, and projection, composition, coalescing, aggregation, resampling and case selection drop it (`maya/resolution/algebra.py`; Section 6.3). No test asserts either behaviour.

Remark 7.4 (Where the valuation lands, and why it cannot be the obvious carrier). Theorem 7.3 is easy to state with $\oplus = \otimes = \max$ over times and nothing else, and the statement is then ill-formed: Theorem 7.5, two subsections below, shows that $(\max, \max)$ over a totally ordered set is not a semiring, and a homomorphism out of $\mathbb{N}[X]$ needs a semiring to land in. The repair is Theorem 7.6 — adjoin an absorbing zero — and it is not a technicality: it is exactly what makes a derived feature with a missing clock report that it has none, rather than reporting the clock of the inputs that happen to have one.

The upgrade is worth stating plainly. “The ingest time of a derived feature is the maximum over its inputs” was documented as arithmetic, on the grounds that arithmetic cannot be forgotten. It is stronger than that: it is the image of the provenance polynomial under a valuation, and a homomorphism has no exceptions to forget. Every route to the number goes through the same object.

Two further questions fall out of the same polynomial:

What does this rest on? The free variables — and these are deliberately *not* the transitive ancestor set, because the two answer different questions. The ancestor walk returns every ancestor, derived ones included, which is the right answer to *what breaks if this changes*. The free variables are the base features and nothing else, which is the right answer to *what data does this ultimately read*. The second is the first minus its derived members, and the identity is asserted rather than assumed: two routes to one answer are worth having only while they agree.

Does it touch the label? Membership of the label’s variable in the free variables. A feature derived from a derivation of the label is still the label, and leakage detection becomes a membership test rather than a graph walk with a depth limit.

In part: “what does this rest on” is answered by MAYA’s upstream lineage walk and by the licence valuation’s walk to leaf sources (Section 7.4); “does it touch the label” is not computed. The recorded challenger flags look-ahead rules and a feature set inheriting its members’ look-ahead (`tests/test_assistant.py::test_a_featureset_inherits_its_members_look_ahead`), which is the non-causality valuation of Section 7.4, not a label-membership test.

7.3 A negative result: currency is not a semiring valuation

The list of semirings above is shorter than the design it came from. That design named a currency valuation, $(\max, \max)$, answering “as of when is this claim current?”. It is not a semiring, and no choice of constants *within its carrier* repairs it.

Proposition 7.5 ($(\max, \max)$ is not a semiring). *Let $(K, \leq)$ be totally ordered and let $\oplus = \otimes = \max$. If $(K, \oplus, \otimes, 0, 1)$ is a semiring then $|K| = 1$.*

Not executed: mathematics only: no test asserts the failure.

Proof. An identity for $\max$ must be a least element, so 0 is the bottom of K ; likewise 1 is the bottom of K ; hence $0 = 1$. In any semiring $0 = 1$ gives $a = a \otimes 1 = a \otimes 0 = 0$ for every a . Independently, the annihilation law fails directly: $0 \otimes a = \max(\perp, a) = a \neq 0$ for $a \neq 0$. $\square$

The practical consequence is specific and worth knowing rather than hiding. Because the zero does not annihilate, a claim resting on a *missing* fact reports the currency of the facts that are present, rather than reporting that it has none. A structure that answers “as of when” for a claim it cannot in fact support is exactly the kind of instrument that reads as reassurance.

It is tempting to conclude that no pair of operations repairs this. That is too strong, and the repair is elementary.

Proposition 7.6 (Adjoining an absorbing zero repairs it). *Let $(K, \leq)$ be totally ordered with a least element $\mathbf{1}$, and let $K_\perp = K \cup \{\perp\}$ for a fresh element $\perp$. Put $a \oplus b = \max(a, b)$ and $a \otimes b = \max(a, b)$ on K , with $\perp$ the identity of $\oplus$ and absorbing for $\otimes$. Then $(K_\perp, \oplus, \otimes, \perp, \mathbf{1})$ is a commutative semiring.*

Proof. Both operations are associative and commutative, $\perp$ is an identity for $\oplus$ and $\mathbf{1}$ for $\otimes$ (on $\perp$ trivially, on K because $\mathbf{1}$ is least), and $\perp$ annihilates by definition. Distributivity $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$ holds on K because $\max$ distributes over itself; if $a = \perp$ both sides are $\perp$; if $b = \perp$ both sides are $a \otimes c$, and symmetrically for c . $\square$

This is the move that makes lineage a semiring in the provenance hierarchy [12]: the zero is a new element meaning *no derivation*, not the least of the old ones. It does not contradict Theorem 7.5, because $\otimes$ on $K_\perp$ is no longer $\max$ with respect to any order in which $\perp$ is least. What it buys is exactly what was missing: a claim resting on a missing fact now evaluates to $\perp$ — “not current at all” — instead of to the currency of the facts that happen to be present. What it does not buy is an aggregate. “How current is this claim, given how many routes support it” is an aggregate over provenance rather than a valuation of it, and aggregates over semiring-annotated data need semimodule structure [15].

In plain terms. One of the six “arithmetics” turned out not to be one. If you use “the later of the two” for both plus and times, then the thing that is supposed to represent *no evidence at all* stops behaving like zero: combine it with a real timestamp and you get the timestamp back, not nothing.

So a claim resting on a missing document reports a date, and the date looks fine. The way to find this is to write the test that asserts the universal property and watch the currency arithmetic fail it. MAYA has no such test, and the same shape is in it three times over (Section 7.4) — which is the argument for the test, made by its absence.

The fix is to invent a new “nothing” rather than borrow the smallest date: then a missing document makes the claim not current at all, which is the true answer.

7.4 Three valuations over the support, and what they do with a gap

MAYA carries no polynomial, but it does carry three computations that are valuations of a derivation’s *support* — its set of leaves — each written directly rather than pushed forward:

Valuation	Combination	Where it runs
Licence terms	most restrictive, per clause	<code>maya/security/licence.py::combine</code> , over the leaf sources of a feature’s derivation and a feature set’s members (<code>maya/services/licensing.py</code>); <code>tests/test_custody.py::test_combination_is_the_most_restrictive_and_remembers_who_imposed_it</code> , <code>::test_derived_works_forbidden_propagates_through_the_algebra</code>
Non-causality	any operand	<code>maya/resolution/algebra.py</code> ; <code>tests/test_resolution_algebra.py::test_non_causality_propagates</code>
Knowledge time	latest member, per row	<code>maya/resolution/featureset.py</code> (feature sets only; Theorem 7.3)

Each of the three uses one operation for both joint dependence and alternatives — a union of two vendor feeds is as restricted as its more restricted feed, exactly as a composition of them is — so each is an instance of the shape Theorem 7.5 rules out as a semiring, and each exhibits the practical consequence that proposition predicts: a leaf with no annotation contributes the *identity*, not a zero. A source that declares no licence contributes no restriction, so a derivation from one licensed and one undeclared source carries the licensed source’s terms alone (`combine` skips it). A member with no knowledge time on a row contributes nothing to that row’s maximum, so the row reports the clock of the members that have one. Both are the permissive direction, and both are the *same* defect, found in three places by one proposition.

Whether that is a defect or a policy is not a mathematical question. Treating an undeclared licence as unrestricted is defensible for data a firm owns outright, and is how most internal data would be registered. But it is a decision that the arithmetic makes silently, and the construction of Theorem 7.6 is what would make it explicit: an undeclared licence as $\perp$ — “terms unknown” — which a derivation then carries forward and an export can refuse by name, instead of an undeclared licence as the most permissive licence there is.

7.5 Citation soundness

Proposition 7.7 (Citation soundness). *Let c be a claim with derivation d_c over evidence identifiers X and let $S \subseteq X$ be a cited set. Define $\nu_S(x) = \top$ iff $x \in S$. Then S supports c if and only if the evaluation of d_c in $\mathbb{B}$ under ν_S is $\top$; equivalently, iff S contains all the variables of some monomial of $\text{eval}_{\mathbb{N}[X]}(d_c)$.*

Proof. By Theorem 7.1, evaluation in $\mathbb{B}$ under ν_S is h_{ν_S} applied to the $\mathbb{N}[X]$ evaluation. A polynomial evaluates to $\top$ under ν_S exactly when some monomial has all its variables in S . $\square$

Not in MAYA: citations are not checked. With no derivation structure there is nothing to evaluate a citation against; the recorded challenger of Section 11 checks a specification document against the formula IR it is bound to, not a claim against its evidence.

This is a small observation with a large practical consequence, taken up in Section 11: over an annotated derivation structure, “does this citation support this claim” is an evaluation, not a further model call.

8 What must still be declared

A paper arguing that four facts derive owes an account of the ones that do not. Three kinds are worth distinguishing, and only the third is irreducible.

8.1 Derived from a declared rule

Observation 8.1 (Constitutive rules). Let $q = f(\Phi)$ be a derived governance quantity: a risk classification, a control requirement, a scope determination. Computing q is deterministic and involves no judgement. The judgement lives in two other places: supplying Φ , and choosing f . And f *constitutes* the standard, so there is no independent specification against which a proposed f could be checked.

Risk tiering is the clearest instance. Let M be a materiality lattice, C a complexity lattice and T a finite chain of tiers.

Proposition 8.2 (Monotone classification). *If $\tau: M \times C \rightarrow T$ is monotone then increasing exposure, criticality of purpose or complexity never lowers the assigned tier.*

In part: materiality tiering, since version 1.0.0: a tier from measured and declared drivers and a firm’s own questionnaire, monotone by construction (Theorem 9.2). The complexity lattice C is not a separate axis: transparency (formula or black box) and the questionnaire’s complexity question stand in for it.

Proposition 8.3 (Control adequacy). *Let Ctrl be a lattice of control sets ordered by strictness and let $\text{req}: T \rightarrow \text{Ctrl}$ and $\text{sup}: \text{Ctrl} \rightarrow T$ form a Galois connection, $\text{req}(t) \sqsubseteq c \iff t \sqsubseteq \text{sup}(c)$. Then “the applied controls meet the tier’s requirements” and “the tier is within what those controls can defend” are the same statement.*

Not in MAYA: as above. The nearest structure is the workflow policy, which states per namespace and per object type which approvals and named checks a transition requires — a req without a τ to feed it.

Monotonicity is trivial as mathematics and consequential as an assurance property: it is exactly the guarantee an examiner seeks — *nothing we can learn that makes a model more consequential will move it into a lighter regime* — and exactly what an unconstrained scoring formula fails to provide. The adjunction means a control deficiency and an inflated classification are one defect seen from two sides. But τ , req and the lattices themselves are declared, and rightly so.

8.2 Declared, but with a consistency oracle

The same artefact may be outside one regime’s governed population, inside a second’s, high-risk under a third and a financial-reporting key control under a fourth. These are not four values of one attribute but four logical systems, each with its own vocabulary and its own notion of what makes a sentence true of an inventory state. Modelling each as an *institution* [20, 21] — signatures, sentences, models, satisfaction — makes admitting a regime the exhibition of a module rather than a migration.

Proposition 8.4 (Determination stability). *For an institution comorphism $\varrho = (\Phi, \alpha, \beta)$ from a regime institution to the core inventory institution, and for every core state M' and regime sentence ϕ ,*

$$M' \models' \alpha_\Sigma(\phi) \iff \beta_\Sigma(M') \models_\Sigma \phi.$$

A determination computed on the core inventory using the translated obligation agrees with the one computed natively in the regime’s vocabulary.

Not in MAYA: regimes. The specification maps MAYA’s evidence onto SR 26-2 and the EU AI Act in prose (§21.3); no regime is encoded, and nothing checks a determination.

Corollary 8.5 (Falsifiable encoding test). *Any observed disagreement between the two sides witnesses a defect in the encoding rather than a genuine normative conflict.*

The encoding is a declaration — a claim about a regulation, not the regulation — but it is a declaration with an oracle attached, which is a strictly better position than a checkbox with a comment. The same shape recurs for deontic consistency: whether a regime obliges and forbids the same term is decidable over the sentences whose form is declared, and the sentences whose form the checker cannot read are *named* rather than assumed consistent, because a check that silently ignores what it cannot judge reports success for exactly the cases it was least able to judge.

Not in MAYA: as above.

A smaller instance of the same shape sits at the edge of the platform. A model registry outside MAYA holds a fact that is declared there: an alias on a model version, which a deployment follows. Whether that version may run is derived here: some execution warrant for it is live. The two can be kept equal without either system trusting the other’s declaration, by a reconciler that reads the derived fact and writes the declared one — sets the alias when a live warrant exists and removes it when none does — and records every change. Between passes the two may disagree for at most the reconciliation period, and a write the registry refuses is reported rather than retried silently. The deployment then follows a declaration that is, up to that bound, a copy of a derivation.

Runs: `maya/services/integrations.py::sync_mlflow; tests/test_integrations.py::test_the_registry_alias_follows_the_warrant`

8.3 Irreducibly declared

Concluding a validation, granting an approval, accepting residual risk, and choosing τ : these are not weakly-checkable tasks conservatively withheld from derivation. They have no notion of correctness independent of the authority exercising them. Section 11 takes up what follows.

8.4 Two constructions that live on the boundary

Contracts. A *contract* (A, G) pairs an assumption on the environment with a guarantee conditional on it [22]. The mapping into the domain is exact and clarifies a term supervisory texts use without defining: a model’s *operating boundary* is A and its performance envelope is G . Use outside the boundary is a contract violation, checkable at execution, and when A fails G is void — a considerably stronger statement than “the input looked unusual”. Substitution becomes refinement, which by Theorem 4.9 is the platform’s one order applied to contracts. A and G are declared; whether one refines another is derived.

In part: the assumption half, as *covenants* on an execution warrant: declared bounds on the inputs (range, null rate, staleness), on the outputs (range) and on use (rows per day). Executions are reported back, the covenants are evaluated, and a breach *suspends* the warrant, so that a consuming call then fails closed, naming whom to contact, until an audited reinstatement (`maya/services/execution.py; tests/test_security_regressions.py::test_each_covenant; tests/test_warrants.py::test_the_checksum_cycle_seal`

`score_execute_and_bundle`). That is “when A fails, G is void” made operational. Refinement between two contracts is not computed, and a copy used offline is labelled *unattested* because nothing can be checked on it. Since version 1.0.0 the assumption half also covers the population a model was fitted on, as a stability covenant, and the reports are read as series and graded (Section 9).

Probe-relative identity, and artefacts that remember. Behavioural equivalence is only as strong as the probe set that witnesses it. Writing Π for a finite probe set and $\equiv_\Pi$ for agreement on it, the three standard version increments have formal content: PATCH asserts the same signature and $\equiv_\Pi$; MINOR the same signature with a different inhabitant of P (Theorem 3.10); MAJOR a different signature. The honest clause is that $\equiv_\Pi$ refines monotonically in Π , so a claim of equivalence is exactly as strong as Π is rich, and a system should record Π with every such claim.

For artefacts that carry state this is not a caveat but a defeater. A stateful kernel $f: P \otimes S \otimes X \rightarrow S \otimes Y$ has, for fixed parameters, a behaviour family $\text{beh}_n: X^{\otimes n} \rightarrow Y^{\otimes n}$.

Proposition 8.6 (Single-shot probes are unsound for stateful artefacts). *There exist stateful kernels v_1, v_2 of the same signature with $\text{beh}_1(v_1) = \text{beh}_1(v_2)$ and $\text{beh}_2(v_1) \neq \text{beh}_2(v_2)$. Moreover, for every n there exist kernels agreeing on all probes of input length at most n and differing at $n + 1$.*

Proof. For the first, let $X = \{*\}$, $Y = \{0, 1\}$; let v_1 emit a fresh fair coin on every call, and v_2 draw one fair coin at start-up and repeat it. Then beh_1 agree and are uniform, while $\text{beh}_2(v_1)$ is uniform on $Y^{\otimes 2}$ and $\text{beh}_2(v_2)$ puts mass $\frac{1}{2}$ on each of $(0, 0)$ and $(1, 1)$. For the second, take $S = \{0, \dots, n\}$ with each call incrementing the state; let both emit 0 while the state is below n , and thereafter let v_1 emit 0 and v_2 emit 1. $\square$

In part: probe-relative agreement, for specification against code rather than version against version. MAYA’s conformance test evaluates the formula IR and the uploaded implementation on sampled inputs — 10,000 by default, drawn from the bound feature set’s own values — and reports the agreement count, the tolerance and concrete counterexample rows, in a sentence that begins “Sampled agreement is not proof” (`maya/formula/conformance.py`; `tests/test_formula.py::test_conformance_finds_counterexample`). Every probe has length one, and MAYA models no state, so Theorem 8.6 is not yet a constraint on anything it checks.

Worked example: a seed policy that was a patch release

A counterparty simulation engine draws random numbers from a generator seeded once at process start. A performance change seeds it per request instead, removing a lock and halving latency. Nothing about the model, the calibration or the parameters changes; the release is labelled a patch.

The nightly suite runs eleven thousand single-trade valuations and reproduces every one within tolerance, because a single valuation averages over paths either way. What changes is the correlation *between* valuations in a batch: two trades in one netting set were previously simulated against common paths and now are not. Netting-set exposure moves materially, in the direction of understatement.

Eleven thousand probes could not detect it, and no number of probes of that shape could have. The defect is not that the suite was too small; it is that every element of it had length one.

9 Judgements over derived facts

Section 8 located the judgement in a governance quantity in two places: supplying its inputs, and choosing the rule that combines them (Theorem 8.1). This section applies that frame to the layer a model-risk function spends its working day in — findings and their remediation, materiality, periodic review, monitoring, the replacement of one model by another, fairness, and the governance of artefacts whose kernel cannot be read at all. MAYA 1.0.0 implements

each of them, and the claim of this section is narrow and checkable: every one is a declared rule applied to facts the earlier sections derive, and none needs a fact to be typed that the platform already holds.

Each judgement also carries a condition on *who* may reach it. We state those conditions beside the mathematics because they are part of what makes the output trustworthy, and because they are the part a register built on declared facts most often leaves to convention.

9.1 Materiality as a monotone map over derived drivers

Definition 9.1 (Drivers and tier). A *driver* is a map s_i from the state of a model to the chain $\{1 < 2 < 3\}$. Let $b \in \{0, 1\}$ be 1 exactly when the model is a declared black box. The *derived tier* is

$$\tau(m) = 4 - \min(3, \max_i s_i(m) + b(m)) \in \{1, 2, 3\},$$

with tier 1 the most material.

Proposition 9.2 (The derived tier is monotone). *If every driver is monotone in the facts it reads, then a change that raises any driver, or that makes a model a black box, never makes the derived tier less material.*

Proof. $\max$ and $+$ are monotone, $\min(3, \cdot)$ is monotone, and $x \mapsto 4 - x$ reverses the order, which is the convention that tier 1 is the most material. The composite is therefore order-preserving from driver scores to materiality. $\square$

This is Theorem 8.2 with its inputs named, and the naming is where the derivation earns its keep. Two drivers are measured rather than declared: *reach*, from the model’s live execution warrants and their execution counts, and *transparency*, from the classification of Section 3 — an opaque kernel is one tier more material than its reach. Two are declared by the owner, use and exposure, and a firm’s own questionnaire contributes the rest, either as the highest-scoring answer or as a sum placed by thresholds. The questionnaire is the declared f of Theorem 8.1; everything it is applied to that the platform can know, it knows. An override is permitted and recorded with its reason, and an override that makes a model *less* material than τ is flagged, because it is the one direction in which monotonicity is a promise the override breaks.

Runs: `maya/services/governance.py; tests/test_governance.py::test_the_tier_is_derived_and_an_override_that_lowers_it_is_flagged, ::test_the_questionnaire_drives_the_tier_and_refuses_answers_it_does_not_offer, ::test_the_points_rule_sums_answers_and_places_them_by_threshold`

9.2 Review as a condition of the licence

Definition 9.3 (Live). For an execution warrant w at time t , let $S(w)$ be its set of suspension reasons. Then $\text{live}(w, t)$ holds when w is sealed, not revoked, t precedes its validity end, and $S(w) = \emptyset$.

A periodic review becomes part of the licence rather than a reminder: when a model’s review falls due, a sweep adds the reason r_{rev} to $S(w)$ for each of its live warrants, and recording a review removes r_{rev} and nothing else.

Proposition 9.4 (Suspensions are independent). *Removing one reason from $S(w)$ never makes live a warrant that another reason keeps suspended, and the order in which reasons are added and removed does not affect live.*

Proof. live depends on $S(w)$ only through emptiness, and set insertion and deletion of distinct elements commute. $\square$

In plain terms. a warrant suspended because a covenant broke stays suspended when a review is recorded. Lifting one kind of suspension never lifts another.

In part: MAYA keeps one reason on a warrant, not a set, and lifts a suspension only when its recorded reason is the review's. The two agree because a warrant that is already suspended accepts no further execution reports, so a covenant breach cannot be recorded over a review suspension or the reverse (`maya/services/governance.py`; `tests/test_governance.py::test_an_overdue_review_suspends_live_warrants_and_a_review_lifts_only_those`). Recording a review is refused to the model's owner.

9.3 Monitoring as covenants read as series

The contracts of Section 8 are evaluated on every reported run. One more assumption is worth naming because it is the one that drifts: the population a model was fitted on. For an input x with baseline proportions b_k over bins fixed from the warrant's own data, and observed proportions a_k over the same bins, the population stability index is

$$\text{PSI} = \sum_k (a_k - b_k) \ln \frac{a_k}{b_k},$$

with each proportion floored at a small ε so that one empty bin does not make the index infinite. It is non-negative, and zero exactly when the two distributions agree on the bins. A covenant on it suspends the warrant; the dashboards read the same reports and grade each warrant on the chain $ok < watch < breach$ as the join of its signals, so a new signal can only raise a grade.

Runs: `maya/services/monitoring.py`, `maya/services/execution.py`; `tests/test_governance.py::test_monitoring_reads_reports_as_series_and_grades_the_warrant`; `case_studies/19-vendor-bureau-score`

Worked example: a bought score that drifted

A vendor's application score is licensed with a PSI covenant on revolving utilisation, the input its own validation found it leaned on most. Over three months the index is 0.009, 0.109 and 0.438: the dashboard grades the warrant *ok*, then *watch*, then *breach*, and the third report suspends it. The scoring service is refused the next morning, a year before a rising default rate would have said the same.

9.4 Replacing a model: a comparison needs a shared escrow

Two models are compared on a holdout. Their two error figures differ, and the question is whether the difference is larger than the noise of which rows happened to be held out. A paired comparison answers it, and pairing is only defined when both errors are indexed by the same rows.

Proposition 9.5 (Equal escrow hashes license pairing). *Let two training warrants escrow holdouts H_1, H_2 , and let h be the canonical content hash, which is sensitive to row order and to every value and insensitive to column order. If $h(H_1) = h(H_2)$ then, up to the collision resistance of h , the i -th error of one model and the i -th error of the other refer to the same row for every i .*

Proof. The hash is computed over the canonical encoding of each row in order, so two tables with equal hashes hold equal rows in equal order unless the hash collides. The errors are computed over the escrowed rows in their stored order. $\square$

Given pairing, MAYA reports the difference in the metric, a percentile interval from a paired bootstrap — rows resampled with replacement, each row keeping its two errors together, with a fixed seed so that the interval is a function of the data — and the share of rows on which the challenger's error is the smaller. The verdict *challenger better* requires the whole interval below

zero. Warrants whose escrow hashes differ are refused rather than compared, and whoever owns the challenger may not record the decision, which is recorded and not enacted: the champion's licences are unchanged until somebody changes them.

Runs: `maya/services/challenges.py; tests/test_challenges.py::test_the_verdict_needs_the_whole_interval_on_one_side, ::test_a_challenger_is_scored_on_the_champions_rows_and_decided_independently, ::test_warrants_on_different_holdouts_are_not_compared`

9.5 A holdout for a series is a suffix, not a sample

The holdout of Theorem 9.5 is drawn by hashing each row's key with the warrant's seed, so it is a sample: every row is equally likely to be held out, whatever its date. For rows that are independent that is the right design. For a time series it fails twice. It trains on the future and tests on the past, which is the look-ahead Theorem 6.6 exists to refuse, arriving by the split rather than by a late record. And it scatters the held-out rows, so a kernel that carries state from one row to the next — a conditional variance, a moving-average error — cannot be evaluated on them at all: the oracle of Theorem 9.8 run over every fifth day is not the model.

A warrant of shape *time series* splits instead by the event date. Let D be the ordered set of distinct event dates of the resolved rows and $0 \leq f_1 \leq f_1 + f_2 \leq 1$ the train and validation fractions; a row with event date e is *train* if e is among the first $\text{round}(|D|f_1)$ dates of D , *validation* if among those up to the $\text{round}(|D|(f_1 + f_2))$ -th, and *test* otherwise, and the rows are kept in event order.

Proposition 9.6 (A series holdout is a suffix). *Under the time-series split, (i) every test row's event date exceeds every train row's; (ii) rows sharing an event date share a partition, so a panel of several series is cut at the same instant for all of them; (iii) the partition does not depend on the seed; and (iv) within each series the escrowed rows are contiguous and in event order.*

Proof. (i) and (ii) hold because the partition is a function of the rank of e in D alone and is monotone in it. (iii) holds because the seed does not enter. For (iv), the test rows of a series are those whose dates are the last dates of D that the series has, which form a contiguous run; the frame is sorted by the index, whose first column is the event date, before it is escrowed. $\square$

Runs: `maya/services/warrants.py::assign_splits; tests/test_warrants.py::test_a_time_series_warrant_holds_out_the_last_dates_in_order`

Theorem 9.5 is unaffected: two warrants of this shape over one pin and one set of fractions escrow the same rows in the same order, so their hashes agree and their errors pair. What changes is what the escrow means. A sample estimates how the model does on rows like the ones it saw; a suffix estimates how it does on the future of the rows it saw, which is the question a forecast is asked.

9.6 Fairness: a summary of error sizes is blind to direction

For a segment s of the holdout with errors $e_j = \hat{y}_j - y_j$, write MAE_s for the mean of $|e_j|$ and β_s for the mean of e_j , the bias.

Proposition 9.7 (Direction blindness). *$|\beta_s| \leq \text{MAE}_s$ for every segment. And for every $m > 0$ there are two segments with $\text{MAE}_1 = \text{MAE}_2 = m$ and $\beta_1 = m$, $\beta_2 = -m$. Consequently no criterion that depends on the segments' error sizes alone can distinguish a model that is equally inaccurate for two groups from one that is wrong for each of them entirely, and in opposite directions.*

Proof. The first claim is the triangle inequality for the mean. For the second, let every error of the first segment be m and every error of the second be $-m$. Both segments have mean absolute error m ; their biases are m and $-m$. $\square$

In plain terms. a fairness check that compares error sizes across groups — the most common one — can report a model that treats two groups identically when it overstates one and understates the other by the whole of its error. MAYA reports bias beside MAE for every segment and marks a segment *systematic* when $|\beta_s| > \frac{1}{2} \text{MAE}_s$: when more than half of its error is one-directional. A segment smaller than a threshold (twenty rows by default) is reported without figures, because a mean over a handful of escrowed rows discloses those rows.

Runs: `maya/services/evidence.py; tests/test_challenges.py::test_segments_suppress_small_groups_and_flag_the_worst; case_studies/45-gompertz-makeham-mortality`

The proposition is elementary and was found the practical way. The first version of the fairness evidence flagged segments on MAE alone, was run on a unisex mortality table — unisex because the law forbids pricing by sex — and reported nothing: the MAE ratio between the sexes was 1.08, while the biases were +0.0259 for women and −0.0240 for men. The correction is the systematic flag. The finding itself is *accepted* rather than fixed, by a manager who is not the model’s owner, with the ruling cited: an instance of the irreducibly declared act of accepting residual risk (Section 8), recorded where an auditor will find it.

Permutation importance — the rise in error when one input is shuffled, seeded and repeated — needs nothing but the model’s outputs, which is what lets it apply unchanged to the next case.

9.7 Oracles, not terms: black boxes and applications

A declared black box has no term in the formula language, so nothing in Sections 4 and 7 can read its kernel. It can still be *run*. If its code artefact has passed validation — parsed, its imports allowlisted, searched for filesystem, network, process and dynamic-code use, and executed twice in a sandbox to check determinism — then the kernel is available as an oracle O_f , and blind scoring computes $\text{metric}(O_f(X_H), y_H)$ with the oracle executed in the sandbox on the holdout’s input columns.

Proposition 9.8 (The target does not reach the oracle). *If the target is not among the columns the model’s input contract names, the sandboxed oracle receives no function of y_H .*

Proof. The sandbox is handed exactly the contract’s columns of the escrowed rows, and it has no network and no view of storage; the target is compared with the oracle’s output outside it. A contract that names the target is refused before anything runs. $\square$

Runs: `maya/services/warrants.py; tests/test_vendor_models.py::test_a_black_box_with_a_validated_artifact_is_scored_blind_in_the_sandbox`

Two details make the oracle usable on series. The contract may name an index column as well as an attribute, because an index column is on every row: a per-series kernel keeps its recursions apart by the series key, and without it a panel’s series would run into one another. And the oracle receives each input as its declared type, a number as a number and a key as text, rather than every column coerced to a float. Neither weakens Theorem 9.8: the target is still an attribute, still outside the contract, still compared outside the sandbox.

Runs: `maya/services/warrants.py::validate_contract; tests/test_warrants.py::test_a_model_may_read_an_index_column_and_a_text_input_is_not_forced_numeric`

An application built on a large language model is the limiting case. Its weights are unavailable; what can be pinned down is the provider, the model name, the system prompt, the prompt template, the sampling parameters and the guardrails. That tuple is the parameter object, inhabited by authorship rather than by fitting, and its identity is a hash of its content.

Proposition 9.9 (Evidence is bound to a definition and an evaluation). *Let an evaluation run e record the definition hash $d(e)$ of the version it scored and the content hash $c(e)$ of the*

evaluation set it used. If approval of version v against set E admits only runs with $d(e) = h(v)$ and $c(e) = h(E)$, then any change to v or to E after the run — to a prompt, a parameter, a guardrail or a single case — leaves v without admissible evidence until it is evaluated again.

Proof. A change to v changes $h(v)$, and a change to E changes $h(E)$, up to collisions; neither changes the hashes already recorded on e . $\square$

Every check is deterministic — containment, a regular expression, a length, a parse — and so is every guardrail: blocked terms, a length cap, and a personal-data scan for e-mail addresses, card numbers that pass the Luhn check, and telephone numbers. No model grades another model, because a judgement the platform cannot reproduce is not evidence it can seal. A version is approved by a model manager who neither owns the application nor submitted the version, and a validator may extend the evaluation set without the owner's leave, which is the owner losing the power to choose what the application is tested on.

```
Runs: maya/services/llm.py; tests/test_llm.py::test_any_change_is_a_new_version_and_an_edited_eval_set_voids_old_evidence, ::test_a_validator_may_add_to_the_evaluation_set_but_a_developer_may_not; case_studies/49-llm-complaint-triage
```

Worked example: right about everything asked, and not fit to use

A complaint-triage application routes twenty-four complaints to the right team, every one of them, and fails three: it promises a refund in one answer and repeats a customer's card and telephone numbers in two. Accuracy on the task — the number a team would naturally report — is perfect; the guardrails make the failures a property checked on every case rather than an impression from a sample. The corrected draft passes; a validator then adds a complaint in Welsh, and the clean run stops counting until the new set is run.

9.8 Findings, and the separation that makes them evidence

A finding is a defect with an owner, a severity and a due date. It is raised by anyone who may read the model, remediated by its owner, and *closed* only by somebody who did not remediate it; the risk may instead be accepted, with a written reason, by somebody other than the model's owner. Each move is kept on the finding and in the audit chain. The rule has no mathematics in it and is included for that reason: it is the same separation that the approval workflow enforces for versions and warrants, applied to the repair, and a register that records findings without it records assertions about fixes.

```
Runs: maya/services/governance.py; tests/test_governance.py::test_a_finding_is_raised_remediated_and_closed_by_someone_independent, ::test_the_owner_does_not_accept_the_risk_in_their_own_model
```

The supervisory inventory is the last judgement and the least interesting mathematically: one row per model and per application, read from the objects above in an SR 11-7 or SS1/23 layout, with a count of the models the exporting user may not see. It is included because it is where the whole argument of this paper is visible to a supervisor. Every column that a typed register would ask somebody to maintain — tier, review dates, open findings, live deployments, monitoring status, restrictions on use — is computed, and the inventory cannot drift from the system because it is the system, read.

```
Runs: maya/services/inventory.py; tests/test_governance.py::test_the_inventory_exports_in_each_layout_and_counts_what_it_leaves_out
```

10 Laws as the criterion of seriousness

10.1 The discipline

A foundation that is documented but unenforced decays into ornament within a small number of releases, and thereafter misleads. The discipline we propose is that each result be restated as an executable law and checked by property-based testing against generated inputs [26], with failure treated as a build failure — and, crucially, that each law be tested *as stated* rather than as the implementation happens to behave. A test written from the code proves only that the code agrees with itself.

10.2 What runs, and what does not

The strongest arrangement available here is a register that is itself executable: one file stating every law, each either asserted against generated inputs or named explicitly as not asserted, with a test that fails when the file and the paper’s table disagree. MAYA does not have one, and the table below is therefore kept in this paper rather than in the system — which is weaker than what this section argues for, by exactly the gap the section is about, and is recorded as such rather than presented as sufficient.

Claim	State	Where, or why not
Model as a parametric kernel (Theorem 3.1)	in part	input roles in the formula IR; no stochastic kernels
Trainability classification (Theorem 3.3)	in part	T0 derived; T6 declared as kind <code>vendor</code> ; the rest absent
Fitting evidence is typed (Theorem 3.4)	in part	T0 only, untested, permissive for models without a closed-form body (Section 3.6)
Fibration, totality, derived base (Theorems 3.5 to 3.7)	absent	nothing is indexed by model kind
Rule-set reachability (Theorem 3.8)	absent	no rule-set models
Refitting preserves the morphism (Theorem 3.10)	runs	version hash excludes parameters; <code>test_hash_ignores_latex_but_not_math</code>
Parameter accumulation (Theorem 3.11)	runs	alias-namespaced composite parameters; <code>test_composite_union_maturity_seeds_and_eval</code>
Change closure (Theorem 3.12)	in part	inside a composite; deprecation notifies warrant owners
The schema order and lattice (Theorems 4.2, 4.6 and 4.7)	absent	only names and datatypes are compared; no join
The partial meet (Theorem 4.8)	runs	composite union contract
One relation, four questions (Theorem 4.9)	in part	three lines, three routines, one disagreement
Missing versus narrowed (Theorem 4.10)	in part	missing and retyped told apart; no bounds
Edit laws (Theorem 4.11)	in part	override-diff inheritance runs; the laws are untested
Composition type-check (Theorem 5.1)	absent	no edges outside composites
No compositional risk (Theorem 5.6)	mathematics	no risk figure; maturity meet over members
The point-in-time operator and its laws (Theorems 6.1, 6.2 and 6.4)	absent	replaced by the cut and the certificate

Claim	State	Where, or why not
Cut, certificate, and their relation (Theorems 6.5 to 6.8)	runs, proved here	the cut and the certificate are tested; the agreement is proved, not tested
Copy, do not connect (Section 6.4)	runs	<code>sql</code> and <code>python</code> sources are pulled
Provenance universality, citation soundness (Theorems 7.1 and 7.7)	absent	no derivation structure
The ingest clock (Theorem 7.3)	runs	feature sets and every algebra operator; <code>test_every_operator_carries_the_knowledge_clock</code>
$(\max, \max)$ is not a semiring; the repair (Theorems 7.5 and 7.6)	mathematics	three support valuations take the identity on a gap (Section 7.4)
Monotone tiering (Theorems 8.2 and 9.2)	in part	derived drivers and a declared questionnaire; no separate complexity lattice
Control adequacy, regimes (Theorems 8.3 and 8.4)	absent	
Operating contracts (Section 8)	in part	covenants, population stability among them, suspend a warrant; refinement not computed
Independent suspensions (Theorem 9.4)	in part	one reason slot, made safe by refusing reports to a suspended warrant
Monitoring grades (Section 9)	runs	<code>test_monitoring_reads_reports_as_series_and_grades_the_warrant</code>
Pairing by escrow hash (Theorem 9.5)	runs	<code>test_warrants_on_different_holdouts_are_not_compared</code>
Direction blindness (Theorem 9.7)	runs	the systematic flag; <code>test_segments_suppress_small_groups_and_flag_the_worst</code>
Target exclusion (Theorem 9.8)	runs	<code>test_a_black_box_with_a_validated_artifact_is_scored_blind_in_the_sandbox</code>
Evidence binding (Theorem 9.9)	runs	<code>test_any_change_is_a_new_version_and_an_edited_eval_set_voids_old_evidence</code>
Independent closure of findings (Section 9)	runs	<code>test_a_finding_is_raised_remediated_and_closed_by_someone_independent</code>
Probe-relative identity (Theorem 8.6)	in part	sampled conformance, length-one probes
Automation admissibility (Theorem 11.2)	runs	the recorded challenger's no-write boundary

Of thirty-four rows, fourteen run, eleven run in part, seven are absent, and two are mathematics that constrains the system without being executed by it. The previous revision had twenty-five rows and six that ran; the difference is the governance layer of Section 9, whose properties were written as tests when the layer was written, and the repair of the ingest clock. A reader is entitled to hold that against the account, and we would rather state the ratio than let it be discovered. The explanation is not a defence: MAYA was specified as a register of features, data and licences to compute, and the constructions it carries are the ones that purpose needed, which is a statement about what was built and not about what the argument requires.

What the system does run, and what no part of the argument above predicted, is a set of laws about the evidence path itself, checked against generated inputs [26]: the canonical encoding is injective up to its documented identifications; a pin's content hash ignores column order and arrival order and notices any changed value; content-defined fragment boundaries tile every table within their bounds, and an inserted row leaves every earlier fragment untouched; a restatement never overwrites (`tests/test_properties.py`). Two further tests hold an

accelerated implementation equal to the authoritative one on randomised inputs (`tests/test_canonical_fast.py`, `tests/test_resolution_grouped.py`). These are the laws that make “the same bytes” a checkable statement rather than an aspiration, and they are the part of this section the implementation honours best.

10.3 What an executable law delivers that a documented one does not

The difference is not diligence. A statement in a document and the code it describes are two accounts of one rule, and two accounts of one rule diverge — not because anybody decides to let them, but because only one of the two is executed, and the executed one is the only one that anything pushes back on. An executable law is the comparison between them, performed by a machine, on a schedule nobody sets.

Four properties follow, and each is a property of the arrangement rather than of the people in it.

1. *A rule stated once cannot disagree with itself.* Theorem 4.9 is the instance, and MAYA is the counter-instance: three routines for fragments of one relation, and a numeric widening in one of them that the other two refuse.
2. *A published contract can be executed rather than quoted.* Section 6.5 is the instance. Where a rule is handed to a system this one does not run, the executable form of the law runs the published rule and compares answers, which is a check on the contract rather than on its spelling.
3. *Two routes to one answer can be required to be the same route.* Independent recomputation is worth its cost only under that requirement, and the requirement is a law rather than a convention.
4. *A property that holds on the examples somebody chose is not the property.* Laws are checked against generated inputs [26], and stated as the law is stated rather than as the implementation behaves — a test written from the code proves only that the code agrees with itself.

The fourth is the one that decides whether the other three are real. It is also the reason the rows above that do not run are listed rather than described: the property they lack is not documentation.

11 Automation is the same boundary

11.1 Oracles

The constructions above were introduced to secure governance properties. They have a second use, which was not the design intent and which we now regard as the most practically consequential: several of them are *decision procedures*, and a decision procedure is exactly what makes machine-generated output safe to accept.

Definition 11.1 (Oracle). An *oracle* for a task T with outputs in O is a decidable predicate $\text{ok}_T: O \rightarrow \mathbb{B}$, computable from the formal structure and from data independent of the output, such that $\text{ok}_T(o)$ holds only if o is correct for T .

Proposition 11.2 (Automation admissibility). *If T is oracle-backed then the soundness of “generate with any procedure g , accept iff ok_T ” is independent of g : no incorrect output is accepted. The generator’s error rate determines throughput, not correctness. If T is not oracle-backed, the correctness of the output is exactly the correctness of the generator.*

Proof. Immediate. In the first case acceptance implies ok_T , which implies correctness by Theorem 11.1, and the rejected fraction is a throughput quantity. In the second there is nothing between g and acceptance. $\square$

The observation this paper adds is that the oracle boundary is not a new boundary. It is the derive/declare boundary of Sections 4 and 6 to 8, seen from the other side: a fact that derives from structure has an oracle, namely the derivation; a fact that constitutes the standard has none, by Theorem 8.1, because there is nothing independent for it to be checked against.

Task	Oracle	From
Encode a regulatory regime	the satisfaction condition	Theorem 8.4
Assert two versions behave alike	$\equiv_{\Pi}$ on the declared probe set	Section 8
Substitute one version for another	$\sqsubseteq$ on inputs, provision on outputs	Theorem 4.9
Wire one model into another	$\text{out}(a) \sqsubseteq \text{in}(b)$	Theorem 5.1
Bind a featureset to a kernel	$S(F) \sqsubseteq \text{in}(k)$	Theorem 4.9
Assemble fitting evidence without leakage	the point-in-time operator, recomputed independently	Theorem 6.2
Cite evidence for a written claim	Boolean pushforward of the derivation	Theorem 7.7
Propose a remediation plan	<i>none needed</i> — computed in the tropical semiring	Section 7
Propose a probe or a query	it runs and discriminates, or it does not	—
Choose the tiering rule τ	<i>none exists</i>	Theorem 8.1
Conclude a validation	<i>none exists</i>	Theorem 8.1
Accept residual risk	<i>none exists</i>	Theorem 8.1

In plain terms. If you can *check* an answer, it does not matter much what produced it: a wrong answer is caught and discarded, and the only cost is wasted effort. If you cannot check it, then trusting the answer is trusting the producer, and no amount of process changes that. So the useful question about automating a governance task is not “is the model good enough?” but “do I have a check?” — and this paper’s answer is that you have a check exactly where the fact was derived rather than declared. The two questions were the same question all along.

The first row inverts an apparent weakness. Encoding a forty-page supervisory statement into signatures and sentences is expensive expert work, and that cost is the practical objection to the institutional approach. It is also a task language models are unusually well suited to, and the output is checkable: a proposed encoding either satisfies Theorem 8.4 or it does not. Generation becomes cheap, verification mechanical, and the expert’s role changes from authoring to adjudicating an encoding that has already passed a consistency test. MAYA encodes no regime, so this row is argument only.

The rows MAYA does carry are fewer, and each is a check on generated output rather than a generator. A formula IR is *lifted* — from LaTeX, from Python by analysis of its syntax tree, from a spreadsheet’s formula graph — and the lift is then checked: a spreadsheet’s lifted IR is evaluated against the workbook’s own cached results and, in a separate test, against LibreOffice Calc recalculating the same workbook headless, and a workbook that disagrees is reported cell by cell and cannot be submitted (`maya/formula/xlsx.py; tests/test_spreadsheet.py::test_a_workbook_that_disagrees_is_reported_cell_by_cell, ::test_a_disagreeing_workbook_cannot_be_submitted; tests/test_spreadsheet_libreoffice.py`). Binding a feature set to a model has the contract check; assembling training data has the leakage certificate; a result handed to a regulator has a bundle whose verifier re-executes it. Spec-code conformance is the instructive exception: sampled agreement can refute an implementation and cannot certify one, so by Theorem 11.1 it is not an oracle, and MAYA’s report says as much in its first four words.

Runs: the recorded challenger (`maya/assistant/`, `maya/services/assistant.py`). On every submission it reads a dossier of the version under review and writes a memo of findings — look-ahead rules, unbounded fills, schema drift, missing limitations, a specification document that disagrees with its IR, licence terms — attributed to the provider and model that wrote it. It never approves, never blocks and never writes to the object under review; the human reviewer records whether they agreed, the author cannot, and someone who cannot approve cannot either (`tests/test_assistant.py: test_submission_queues_a_memo_that_cannot_block_or_edit, test_the_reviewer_records_a_response_and_the_author_cannot, test_someone_who_cannot_approve_cannot_respond`). The default provider is a deterministic rule set; the Claude provider is opt-in and has been tested only against a stub of its client, so whether the live service answers as the stub does is unproven.

That is the boundary of Theorem 11.2 drawn at its most conservative. The assistant’s output is not accepted by anything — it is placed beside a decision that remains a person’s — so its correctness is never load-bearing, and the only thing the system must guarantee is the no-write boundary, which it tests.

11.2 The reviewer is part of the system

Theorem 11.2 guarantees that no incorrect output is accepted *by the oracle*. In a governance process acceptance is an act performed by a person, and the composite that actually runs includes them. The uncovered part has a documented failure mode: machine-drafted text is fluent, fluency is read as care, and reviewers of polished artefacts find fewer defects than reviewers of rough ones. We cannot dissolve this, and the framework does three things short of dissolving it.

It determines the division of attention formally. Where an oracle exists, the reviewer is not checking correctness — Theorem 11.2 disposed of that — but authority, which by Theorem 8.1 admits no oracle. Marking that boundary in the interface is a mitigation available only because the boundary is formally determined rather than a matter of taste.

It supplies constructed rather than rhetorical objections. An uncited claim is a failing conjunct under Theorem 7.7. A refused wire names the field that does not compose. A featureset that does not serve a kernel names the slot that is missing or narrowed (Theorem 4.10). These survive being stated in prose as fluent as the draft’s, because none of them is a matter of emphasis.

It makes one thing measurable that otherwise is not. A claim in a generated document is supported by a set of nodes in the derivation structure. Whether those nodes were retrieved during the review is a fact about the session rather than an inference about the reviewer, and a pattern of attestations to claims whose support was never opened is the clearest available indicator of the failure mode in question. The sharpest signal is the rate at which a reviewer accepts drafts the oracle refused: exactly the population where person and procedure disagree, and a quantity that exists only in a system with a procedure to disagree with.

Remark 11.3 (Status). None of this has been evaluated. These are hypotheses the framework generates rather than results it establishes, and two of them are unavailable to a governance system that does not represent evidence as a derivation structure — which is a testable claim about the framework’s usefulness, recorded in Section 14 as one of the things that would falsify it.

11.3 Stratification

If machine assistance is admitted into a system that governs machines, one should ask whether the account is well-founded. It is, by stratification rather than by good intentions. Let $\mathcal{G}$ be the governing machinery — the classification map, the derivation structure, the institutions, the order — and $\mathcal{A}$ the governed population. An assistant is registered as an element of $\mathcal{A}$: it has a parameter object, a fitting morphism (configuration), a contract, a classification and evidence. No element of $\mathcal{G}$ is an element of $\mathcal{A}$, and no construction here requires a fixed point.

Not in MAYA: registering the assistant as a governed model. The challenger’s memos are attributed to the provider and model version that wrote them, which is the record the stratification needs; the assistant itself is configuration, not an entry in the register it reviews.

One dependency crosses the strata and is worth naming rather than hiding: if an assistant drafts a regime encoding, part of $\mathcal{G}$ has been produced with help from an element of $\mathcal{A}$. The resolution is that the dependency is mediated by an oracle which is itself not machine-produced — the satisfaction condition is a property of institutions, and it is evaluated mechanically. *Generation may cross the strata; acceptance may not.*

12 The implementation

12.1 What it is

MAYA — Model & AI Lifecycle Assurance — is a platform written by the author to the account above and released as the artefact accompanying this paper, at <https://github.com/ajsinha/maya>. It is a system of record for quantitative features, feature sets and models, and for *warrants*: a training warrant freezes a model version against a feature-set version and receives the parameters training produced; an execution warrant licenses a model, its parameters and its input contract to run, and can be suspended or revoked while in use. It does not train models and does not serve predictions. The boundary is deliberate and is the subject of Section 6.5: a platform that executed the artefacts it governs would be checking its own work.

It reaches the edge of that boundary in five places, and each was built to stay on this side of it. An MLflow alias follows the licence, as Section 8 describes. A guard in the SDK checks the warrant before a scoring call and reports the run after it, from inside the caller’s own process. A training warrant can be turned into a signed job definition for the firm’s own compute, with a key that expires in a day; the job runs there, and its parameters come back through the checksum cycle like any other. For a closed-form model the platform can fit the parameters itself on the training rows, as a second route to the same answer recorded as evidence and never as a parameter set: it checks a fit and does not supply one. And under a live warrant it can score a pinned table, running only what blind scoring already runs — the formula from its term, a black box as the validated oracle — with the output sealed by its content hash, the run reported so its covenants are evaluated, and the pin and the output hash entered in the warrant’s custody chain.

```
Runs: maya/sdk/guard.py, maya/services/training_ops.py, maya/services/batch_scoring.py;
tests/test_integrations.py::test_a_guarded_scoring_call_is_licensed_reported_and_refused_
once_suspended, tests/test_challenges.py::test_a_dispatched_job_fits_on_its_own_compute_and_
reports_back, ::test_a_reference_refit_finds_the_true_parameters_and_says_whether_a_set_
agrees, tests/test_warrants.py::test_batch_scoring_is_attested_reported_and_refused_off_
warrant
```

At the revision this text describes — version 1.0.0 and the work since, described by revision 2.7 of its own specification — it is 216 Python modules in `maya/` and 11 more in `maya_delta/`, its own implementation of the Delta Lake protocol; one typed metadata from which the SQLite and PostgreSQL schema files are generated and checked for drift; 252 HTTP operations under a locked API snapshot, each with an SDK method and held one-to-one by a gate that fails the build on a mismatch; and 2,109 tests. The suite runs on Linux over SQLite; it last ran over PostgreSQL 16, 17 and 18 before the governance layer of Section 9 was added, which has run on SQLite only. The case studies have been run on Windows, where the sandbox has no operating-system isolation, but Windows is outside the test matrix and macOS has not been tried.

Four measured results bound what it is, all from the repository’s `docs/BENCHMARKS.md` with the unedited result files beside it. Resolving a feature set of 500 symbols by ten years of business days by 50 attributes takes a warm p95 of 0.98 s, and 6.1 s with a bounded forward

fill on every attribute, against a target of 15s. Catalog search over 100,000 objects has a p95 of 0.16s. The worst metadata page has a p95 of 0.15s on SQLite with one process and 0.18s on PostgreSQL with eight. The fourth target — 200 concurrent interactive users on one node without p95 degradation — is *not met reliably*: three runs gave 0.34s, 0.22s and 0.43s against a 0.3s target, on a shared workstation rather than a benchmark host. One run of three passing is not a pass, and the repository says so.

Construction	Module	What it refuses
The cut and the certificate (Section 6.3)	<code>maya/resolution/resolver.py</code> , <code>maya/services/warrants.py</code>	a training warrant whose rows were learned after their decision date, unless each exception is justified in writing
The partial meet (Theorem 4.8)	<code>maya/formula/composite.py</code>	a composite whose members want one attribute at two datatypes, naming both
Parameter accumulation (Theorem 3.11)	<code>maya/formula/evaluate.py</code>	nothing; it is how a composite’s one parameter set is read
Copy, do not connect (Section 6.4)	<code>maya/services/sources.py</code>	a read-through to a live source: SQL and Python sources are pulled into the ingest log
Support valuations (Section 7.4)	<code>maya/security/licence.py</code> , <code>maya/resolution/algebra.py</code> , <code>maya/resolution/featureset.py</code>	an export, grant or derivation that breaches a source’s terms, naming the source and the clause
The no-write boundary (Section 11)	<code>maya/assistant/</code>	the assistant approving, blocking or editing anything
Materiality, review, findings (Section 9)	<code>maya/services/governance.py</code>	an owner reviewing their own model, closing their own fix, or accepting the risk in it
Pairing (Theorem 9.5)	<code>maya/services/challenges.py</code>	a comparison of warrants with different escrows, and the challenger’s author deciding
Oracle scoring (Theorem 9.8)	<code>maya/services/warrants.py</code>	a black box without a validated artefact, and a contract that names the target
Evidence binding (Theorem 9.9)	<code>maya/services/llm.py</code>	an application submitted without a clean run on its exact definition and evaluation set

12.2 What building it changed

An implementation earns its place in a paper by changing the paper. Nine places where it did, each of which is already in the text above rather than confined to this section.

The certificate replaced the operator. Theorem 6.1 is the operator; MAYA reads with one scalar knowledge bound, as most bitemporal systems do, and then *certifies* the per-row bound on the training set, signs the result and attaches it to the warrant. Theorems 6.7 and 6.8 are what that arrangement is worth, and they were proved to find out rather than designed in: agreement with the operator holds exactly where the certificate is clean, and saturation survives as “reproducible or refused”. The arrangement buys something the operator cannot

express — a late fact accepted with a written reason, which a validator sometimes needs — and costs the guarantee on every row the certificate cannot see. Neither half was visible from the mathematics alone.

A contract that was true of everything. A declared black-box model was found to carry an empty input contract, so every feature set satisfied it; it now carries the contract it declares (`tests/test_vendor_models.py::test_vendor_warrants_behave_as_internal_ones`). The same shape sits one step later and is unrepaired: a model without a closed-form body, or a warrant naming no parameter set, passes the parameter check as “non-trainable” (Section 3.6). Both are D2’s permissive fallback — a check that reads an absence as compliance — and neither was found by review. They were found by asking what the check does when the structure it reads is missing, which is the question Theorem 3.7 says to ask.

Two implementations, one authority. Speed was wanted in three places where a second implementation was the obvious route: the lake (a native backend and a pure-Python one), the canonical encoding behind every content hash (a column-at-a-time encoder, six to eight times faster), and the fill rules (vectorised over whole columns, which took the forward-fill benchmark from 156s to 6s). In none of them is the second implementation trusted to agree. For the canonical encoding and the fill rules the original is named the authority and the fast one is held equal to it on randomised inputs — an accelerator that disagrees with its reference is by definition the one in error. For the lake, both backends answer to one conformance suite and must read each other’s tables. That is the moral of Section 4 applied to performance rather than to semantics, and it is the part of this paper’s discipline the implementation applies most consistently.

Reproducibility by hash, not by bytes. A namespace chooses whether a feature-set pin writes its resolved output (always, the default) or is sealed by the hash of that output and replayed from its member pins on every read (`on_demand, never`). All three seal the same hash (`tests/test_materialization.py::test_every_policy_seals_the_same_content`). The second arrangement makes reproducibility a property of the resolver rather than of storage, which is the stronger claim and the more fragile one: a replay differing by one byte is refused rather than served (`::test_a_replay_that_does_not_reproduce_the_seal_is_not_served`) — safe, and then unavailable. It is the trade of Theorem 6.4 made in bytes, and a reader who prefers availability should prefer the default.

The clock the algebra dropped. Theorem 7.3 says a derived feature’s knowledge time is the latest of its inputs’. Checking the claim against the code found that feature-set assembly honours it and six of the fifteen feature-algebra operators do not: projection, composition, coalescing, aggregation, resampling and case selection returned frames with no knowledge-time column, so the certificate of Theorem 6.6 examined none of the rows of a feature derived through them. The previous revision of this paper reported that and did not repair it. It is repaired: each operator now carries the latest knowledge time of the input rows it was built from, and a test asserts it for all six (`tests/test_resolution_algebra.py::test_every_operator_carries_the_knowledge_clock`). It remains the clearest example in this work of what the markers are for: the proposition was stated, the implementation contradicted it, the two sat on the same page, and the page is what got it fixed.

A black box refused, then run. The previous revision recorded as a design decision that a declared black box is refused at blind scoring, because the platform evaluates formulas and a black box has none. That was the right refusal for the wrong reason. The platform cannot

evaluate such a kernel, but it can *run* one whose artefact it has validated, as an oracle in its sandbox, on the escrowed inputs alone (Theorem 9.8). The distinction between a term the platform can read and an oracle it can only execute is now drawn in the text, and a bought score is validated on the firm’s own outcomes rather than not at all.

A fairness check that saw nothing. The first fairness evidence flagged a segment whose MAE was far above the overall figure. Run on a unisex mortality table it flagged nothing, because men’s and women’s errors were the same size in opposite directions. Theorem 9.7 states why no size-only criterion can see that, and the systematic flag is the repair; neither was anticipated before the case study ran.

Every split was a sample. Until a time-series study was built, every warrant’s holdout was a hashed sample of rows, and nothing in the account said that is wrong for a series. Building the study found it, and Theorem 9.6 is the repair: a warrant of shape *time series* holds out the last dates, in order.

A feature that hid its own columns. A feature’s declared schema describes what is uploaded, and a lag or a derived column exists only after the feature’s transforms run. The metadata a feature reported was the declared schema, so a feature set could not map a lag that every row of the pin held. The feature now reports its schema as the transforms leave it, which is the output-schema function the transform module always had and the resolver did not use (`tests/test_warrants.py::test_a_features_transforms_are_part_of_what_it_offers`). It is the same lesson as the dropped clock above: a derivation that a module can compute and that its caller does not ask for is, to everything downstream, absent.

12.3 Fifteen models, carried the whole way

The repository carries fifteen worked case studies, each a model taken through the whole chain by scripts that use the platform’s own SDK, and each run from nothing by the test suite: a retail PD scorecard, a mortgage cashflow model and a prepayment model, a home-equity exposure model, a Black–Scholes pricer, an IFRS 9 expected-credit-loss composite, a neural network governed as a declared black box, a factor model, a Nelson–Siegel yield curve fitted from LaTeX the quant wrote, and five added with version 1.0.0 — a Basel IRB capital formula proved by reconciliation, a bought bureau score imported from MLflow and scored in the sandbox, a demand model replaced by a challenger on a paired comparison, a unisex mortality table whose bias by sex is accepted in writing, and a language-model complaint-triage application, and since then an AR(2) mean with a GARCH(1,1) variance on daily index returns (`case_studies/`). They exist because a formalism that has only been exercised by its author’s unit tests has not been exercised.

Two of them earn a mention here for what they refused to do. The expected-credit-loss study is a composite of three fitted members under one combiner whose two parameters — a significant-increase threshold and a lifetime multiple — belong to the accounting committee and to no member; it is the worked instance of the union contract of Theorem 4.8 and of parameter accumulation (Theorem 3.11). Scored against its own generating process it produces a portfolio allowance 2.67 times the realised loss, and a sensitivity grid shows the committee’s threshold explains part of that gap and not the rest of it. The study records the finding as open rather than choosing the threshold that would have closed it. The Nelson–Siegel study registers the curve from four lines of LaTeX and then runs into the constraint the paper’s order cannot express: $\beta_0 + \beta_1 > 0$, a non-negative instantaneous short rate, is a statement about two parameters at once, and a table of per-parameter bounds has no place to put it. Joint constraints were added

to the formula IR because that study asked for them (`maya/formula/ir.py: constraint_errors`), which is the smallest possible instance of an implementation changing an account.

The time-series study is the most recent instance. Its mean model is a formula once its lags are transforms on the governed feature; its variance model carries yesterday’s variance as a state no column holds, so it is a declared black box, run as an oracle in the sandbox. Its stationarity conditions, $\phi_1 + \phi_2 < 1$ and $\alpha + \beta < 1$, are joint constraints of exactly the Nelson–Siegel kind, and a persistence of 1.05 offered with each coefficient inside $[0, 1]$ is refused at upload. It is also the study that found the three gaps above that assumed rows are independent: the random split, the hidden columns, and a contract that could not name the series key.

12.4 What the implementation does not establish

It shows that parts of the account are computable, that they refuse real inputs, and — in the certificate and the evidence-path properties — that they can be checked rather than asserted. It does not show that a system built this way is easier to construct, extend or operate than one built without it. There is no comparative study, no user evaluation and no deployment at a supervised institution; the specification’s own usability criterion, a new model designer publishing a first model unaided in under an hour, has not been measured with a real person. A large test suite is evidence that a thing is internally consistent, not that it is useful.

And the system is written by the author of the paper, so it is a demonstration of sufficiency and not an independent replication. The most useful thing a reader could do with it is to disagree with it in public: it is on GitHub, the register of Section 10 can be read against it in an afternoon, and every marker in this paper names a path.

12.5 Artefact availability

The platform described here is at <https://github.com/ajsinha/maya>, and it is the artefact this paper should be judged against. Every `Runs` marker above names a module and a test as paths from that repository’s root, so any claim in this paper can be checked by opening two files. The register of Section 10 lists all thirty-four and can be walked in an afternoon.

What a reader will find alongside the code: the specification the system was built to, with its own revision history; the measurement results behind every number in this section, unedited, with the scripts that produced them; the fifteen case studies as runnable scripts; a quick start and a REST API guide whose examples the test suite executes; and the decision records for the choices a reader is most likely to disagree with — among them why there are no migrations and two generated schema files instead, and why the platform will not execute the artefacts it governs except as a sandboxed oracle on escrowed inputs.

12.6 For the practitioner

The mathematical audience and the model-risk audience are close to disjoint, and a paper that serves both usually serves neither. MAYA carries a second register of some of this content for the second audience: help topics on bitemporality, pins, warrants, licences and conformance, with tutorials and reference guides written with no notation at all (`maya/web/guides/`, `maya/web/templates/help/`). They describe the system and not this account, and a reader who wants to know what the platform does rather than why should start there.

13 Related work

Categorical foundations for learning and probability. The **Para** construction and its use in gradient-based learning is due to Fong, Spivak and Tuyeras [4], elaborated by Cruttwell

and collaborators [5] and situated in a wider setting by Capucci and collaborators [6]. Our use is orthogonal: we are not concerned with how parameters are optimised, but with the fact that a parameter object exists and may be inhabited by procedures other than optimisation — which is what makes Theorem 3.3 a classification of the population rather than of a technique. Markov categories are due to Fritz [1] and Cho and Jacobs [2]; we use them for one specific reason, which is that copying is explicit and Theorem 5.7 needs it to be.

Order, subtyping and variance. Theorem 4.6 is elementary lattice theory [7, 8], and Theorem 4.2 is the record-subtyping order with contravariant inputs [9] in the behavioural sense of [10]. We claim no novelty in the order. What appears not to have been done is to identify the four governance questions of Theorem 4.9 as that one order, and in particular to use the *meet* — rather than a pairwise compatibility check — as the answer to whether one feature set can serve several models, with the partiality of the meet (Theorem 4.8) as an informative refusal.

Feature stores and semantic layers. The engineering prior art for Section 6 is substantial and largely uncited by the governance literature. Feature stores — Feast [29], Tecton [30], Hopsworks [31] — provide point-in-time correct joins, feature versioning and online/offline consistency, and their treatment of the two clocks is essentially the one we formalise. Semantic layers — dbt [32], Cube [33], Malloy [34] — provide the definition-once, compose-many discipline that Theorem 4.11 makes algebraic. What these systems do not supply, and what this paper adds, is the operator formulation with the saturation law (Theorem 6.4): the reproducibility guarantee is usually described as a property of a correctly written pipeline rather than as a property of the read, and the difference shows up precisely in the choice of $\min(\ell, a)$ over a — the previous easiest thing to get wrong, and the choice MAYA makes deliberately and then checks (Section 6.3). A signed certificate of the check is, as far as we know, not something these systems issue. Neither family represents materiality, approved use, independent challenge or remediation, which is the other half of the division this paper’s introduction describes.

Provenance. Semiring provenance is due to Green, Karvounarakis and Tannen [11, 12], with the hierarchy of [13, 14]. Existing applications are to query answering. Ours are to assurance evidence, where the derivations are governance inferences, and to derived features, where Theorem 7.3 turns a pipeline convention into a homomorphism. Our negative result (Theorem 7.5) is the elementary observation that a currency valuation is an aggregate rather than an annotation, which is the difficulty [15] addresses properly; its repair as a valuation (Theorem 7.6) is the adjoined zero that makes lineage a semiring in that hierarchy [12], and we claim nothing for it beyond noticing that Theorem 7.3 needs it.

Bitemporal data. Snodgrass [16] is the standard reference; SQL:2011 brought system- and application-time periods into the standard [17]. Leakage as a machine-learning failure mode is documented by Kaufman and collaborators [18]. The contribution here is not the two clocks but Theorem 6.2, and the relation between it and a certified scalar cut (Theorem 6.7).

Indexed families. Fibrations and the Grothendieck construction are standard [19], and Theorem 3.5 is an unremarkable instance. What we have not seen stated is Theorem 3.7: that an indexed family of *obligations* carries a constraint on its base which an indexed family of mathematical structures does not, because the totality of the family is a property somebody will rely on, and a base whose values are supplied rather than computed cannot support both totality and free extension. The constraint is invisible in the mathematics and decisive in the application, which is the general shape of what this paper is trying to do.

Specification over multiple logics, contracts, abstraction, lenses. Institution theory is due to Goguen and Burstall [20, 21]. We have not found prior application to overlapping normative regimes over a shared asset inventory, nor use of the satisfaction condition as a consistency test for regulatory encodings — but we hold that weakly. Multi-jurisdictional compliance, legal knowledge representation and normative-systems modelling are large literatures with their own institutional and deontic apparatus, and a reader who knows of the corresponding construction there should read our claim as *we did not find it* rather than *it is not there*. Assume-guarantee contracts are due to Benveniste and collaborators [22, 23]; we import the algebra unchanged and observe that operating boundaries and performance envelopes are exactly assumptions and guarantees. Sound abstraction [24] supplies the criterion for when a summary document is trustworthy, and well-behaved lenses [25] the discipline for regenerating one; both are stated here and neither is implemented in MAYA.

Documentation and metadata practice. Model cards [27] and datasheets [28] specify *what* a summary should contain; sound abstraction contributes a criterion for *when* a summary is trustworthy, independent of its contents. Registry and lineage systems address a subset of this territory but assume a homogeneous population of learned artefacts, which is the assumption Theorem 3.3 removes.

Model inventory and model risk platforms. The commercial category this paper is about is real and we should say what it does. Enterprise model risk platforms maintain an inventory, a tiering questionnaire, an approval workflow, a validation record and a findings register; ML platforms maintain experiments, a model registry, lineage, deployment and monitoring. Both are mature. The division is consistently the same one: the first family holds materiality, approved use, independent challenge and remediation but cannot see the artefact, and the second holds the artefact but has no representation of any of the four. Neither family is deficient at what it does, and the point of Theorem 3.3 is not that they should merge but that the merged thing needs a definition of “model” that neither currently has — one under which a closed-form pricer, an elicited weight scheme and a foundation-model application are instances rather than exceptions. We are not aware of a platform in either family that derives the kind of an artefact rather than recording it; MAYA does not either, beyond the row that separates a model with parameters from one without.

Dependency-aware risk aggregation. Theorem 5.6 should be read against an existing literature and not as a discovery. Concentration risk, common-factor exposure and model interdependency analysis are standard in quantitative risk, and supervisory texts have asked for aggregate assessment accounting for shared assumptions, data and methodology since 2011 [39, 40, 41]. Practitioners know aggregate model risk is not additive. What is contributed here is the reason: the obstruction is the comonoid copy map, so it is structural rather than a modelling difficulty that a better correlation assumption removes — and consequently a sharp statement of the shape any correct aggregate must have, which is a constraint on candidate measures rather than a measure. Whether that constraint is worth its formalism against the existing concentration analyses is an empirical question we have not settled, and Section 14 says so.

Multi-jurisdictional compliance and normative systems. Overlapping regulatory scope over one population is not an unstudied problem. Deontic logic, normative multi-agent systems, legal knowledge representation and regulatory-technology work on cross-border obligation all address versions of it, with their own apparatus for conflict, derogation and jurisdiction. Our claim is narrower than it may read: we use institutions [20] because the *satisfaction condition*

gives a falsifiability test for an encoding — truth must be invariant under translation, so an encoding that fails it produces determinations nobody can defend — and we have not found that particular use elsewhere. A reader who knows of it should read Section 8 as one construction among several rather than as the first.

14 Limitations and falsification

No implementation *study*. Section 12 describes the implementation, which is not evidence of the claim that matters. A large test suite shows a thing is internally consistent; it does not show that a system built on this foundation is easier to construct, extend or operate than one built without it. There is no comparative study, no user evaluation and no deployment at a supervised institution, and the implementation is written by the author of the paper — so it demonstrates sufficiency and not independent replication. That remains the principal missing evidence, and adding modules does not reduce it.

The derivations still read declarations. Section 3.6 states this for the trainability class, and the point generalises: the order compares declared schemas, the operator reads declared clocks, the polynomial is built over declared derivation edges. The claim is that the declared surface shrinks and becomes checkable, not that it vanishes. Where the fallback direction of a derivation is permissive — as it is for an artefact whose fitting procedure was never recorded — the old failure mode returns in a smaller place. MAYA shows it returning in three: the parameter check that reads a missing body as “non-trainable” (Section 3.6), the certificate that cannot see a row with no clock (Section 6.3), and the support valuations that read a missing annotation as the identity (Section 7.4).

Three routines for one relation. Theorem 4.9 argues for one implementation of fit, and MAYA has three, which already disagree on whether an integer may stand in for a float. Theorem 5.3 records a second divergence the implementation has not yet reached: composition and version substitution may judge output narrowing differently. Both readings are defensible; holding both silently is not.

Most of the core account does not execute. Of the thirty-four rows of Section 10, fourteen run, eleven run in part and seven are not in the system at all, and the rows that run are concentrated in the governance layer rather than in the order and the operator; the register is kept in this paper and not in a test, so nothing enforces that it stays true. In particular Theorem 5.8 — the interaction premium, which we present as one of the paper’s more useful consequences — is not computed anywhere. Theorem 5.6 says what a correct aggregate cannot be, and does not construct one.

The lattice is finite-fragment, and half of it is not built. Theorem 4.6 gives no bottom element, and the meet is partial. Both are honest and both mean the structure is weaker than “schemas form a complete lattice” would suggest. In MAYA the meet is in use, on names and datatypes, and the join and the order’s bounds and nullability are not built. A reader should take neither the availability of an operation nor its absence from one build as evidence about the mathematics.

The judgements are only as good as their rules. Section 9 shows that tiering, monitoring and fairness are computed from derived facts; it does not show that a firm’s questionnaire, its PSI thresholds or the systematic cut-off of one half are the right ones. They are declared rules in the sense of Theorem 8.1, chosen for being conventional and stated for being contestable.

The evaluation checks for language-model applications are deterministic by design, which makes them reproducible and also means they test what can be written as a check and nothing subtler; and the platform’s connectors to external model registries have been tested against the documents those systems publish, not against live services.

Formalising normative text is lossy and contestable. Encoding a regime as an institution commits to a reading of prose that is deliberately open-textured. The encoding is a claim about the regulation, not the regulation, and different institutions may reasonably encode the same text differently. The formalism makes the claim explicit, citable and testable for internal consistency; it does not make it correct.

Oracles are necessary, not sufficient. Theorem 11.2 says nothing about what an unchecked output does to the humans around it, and the instruments of Section 11.2 are hypotheses we have not evaluated.

The oracle boundary may be drawn too conveniently. Theorem 8.1 classifies the decisions that matter most as admitting no oracle, which is a comfortable conclusion for anyone who would prefer humans to retain them. We believe the argument is sound — a rule that constitutes a standard cannot be checked against that standard — but note that it is exactly the sort of argument whose conclusion should be scrutinised in proportion to how much one likes it.

What would falsify the account. Five things, and Section 12 moves none of them — which is worth stating, because an implementation is easily mistaken for an answer to criticism it does not address. First, an artefact class in the domain that cannot be presented as a (P, φ) pair without distortion. Second, a governance question about fit that is genuinely not an instance of $\sqsubseteq$ — which would show Theorem 4.9 to be a coincidence of four cases rather than a unification. Third, a fragility-sensitive aggregate risk measure that is nonetheless compositional, which Theorem 5.6 forbids, so a purported example would indicate an error in our failure semantics. Fourth, and most likely, a demonstration that practitioners cannot use the derivations the formalism produces — that a refusal naming a slot with no meet is no more actionable in practice than a flag with a comment. Fifth, a finding that the review instruments of Section 11.2 do not move the fluency effect. The fourth and fifth are empirical questions about human factors and we cannot settle either from the armchair.

15 Conclusion

Model governance is treated as an engineering problem with a compliance flavour. We have argued that it is better understood as a problem with a missing foundation, and specifically that its characteristic failures are declarations standing where derivations belong.

Four of those declarations do not need to be made. What kind of artefact this is derives from how its parameter object is inhabited, which makes a closed-form pricer a type rather than an exception. Whether one thing fits where another fitted derives from a single partial order, whose meet answers a question that four separate implementations could not even ask, and whose failure to have a meet is the honest answer to whether one feature set can serve two models. What a training row could have known derives from an operator whose fourth property — saturation at the label — *is* reproducibility, and is supplied by a min that is easy to omit and whose omission is invisible. What a claim rests on derives from a polynomial, and so does the ingest clock of a derived feature, which upgrades a rule somebody could forget into a homomorphism that has no exceptions.

We have been equally concerned to say what does not derive. Aggregate risk cannot be compositional, and the obstruction is the copy map. Currency over a totally ordered carrier is not a semiring valuation, and is one only once “no support” is given a value of its own — a repair three computations in the implementation still lack. Most of the account does not execute in that system, and this revision says which parts. And some facts — the tiering rule, the regime encoding, the approval — constitute the standard they would be checked against, and are therefore irreducibly declarations made by somebody accountable.

That last boundary turned out to be the useful one. Having drawn it for governance reasons, we found that it answers a question we did not set out to ask: *where may a machine do this work?* The criterion is not the model’s capability nor the task’s sensitivity, but whether the fact was derived or declared. Where it derives, generation is free because acceptance is checked. Where it constitutes the standard, there is nothing for the automation to be checked against, and the question is not one of risk appetite but of category.

The judgements a model-risk function makes every day fall on the same line. A tier, a review date, a monitoring grade, a verdict between two models, a fairness flag and an application’s approval are each a declared rule applied to facts the platform derives, and the properties that make them trustworthy — monotonicity, pairing by a shared escrow, sensitivity to the direction of an error, evidence bound to exactly what it was gathered on — are as elementary as the rest, and as easy to leave out.

None of the mathematics is new. What we suggest is that a domain served by tools that each solve half the problem is not waiting for a better product but for a definition — and that the definition, once written down and made executable, pays for itself twice.

References

- [1] T. Fritz. A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370:107239, 2020.
- [2] K. Cho, B. Jacobs. Disintegration and Bayesian inversion via string diagrams. *Mathematical Structures in Computer Science*, 29(7):938–971, 2019.
- [3] M. Giry. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics 915, Springer, 1982, pp. 68–85.
- [4] B. Fong, D. I. Spivak, R. Tuyeras. Backprop as functor: a compositional perspective on supervised learning. *LICS*, 2019.
- [5] G. S. H. Cruttwell, B. Gavranović, N. Ghani, P. Wilson, F. Zanasi. Categorical foundations of gradient-based learning. *ESOP*, 2022.
- [6] M. Capucci, B. Gavranović, J. Hedges, E. F. Rischel. Towards foundations of categorical cybernetics. *Applied Category Theory*, EPTCS, 2021.
- [7] B. A. Davey, H. A. Priestley. *Introduction to Lattices and Order*. 2nd edition, Cambridge University Press, 2002.
- [8] G. Birkhoff. *Lattice Theory*. 3rd edition, American Mathematical Society Colloquium Publications 25, 1967.
- [9] L. Cardelli, P. Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [10] B. H. Liskov, J. M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.
- [11] T. J. Green, G. Karvounarakis, V. Tannen. Provenance semirings. *PODS*, 2007.

- [12] T. J. Green, V. Tannen. The semiring framework for database provenance. *PODS*, 2017.
- [13] P. Buneman, S. Khanna, W.-C. Tan. Why and where: a characterization of data provenance. *ICDT*, 2001.
- [14] J. Cheney, L. Chiticariu, W.-C. Tan. Provenance in databases: why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [15] Y. Amsterdamer, D. Deutch, V. Tannen. Provenance for aggregate queries. *PODS*, 2011.
- [16] R. T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000.
- [17] K. Kulkarni, J.-E. Michels. Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43, 2012.
- [18] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman. Leakage in data mining: formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.
- [19] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141, Elsevier, 1999.
- [20] J. A. Goguen, R. M. Burstall. Institutions: abstract model theory for specification and programming. *Journal of the ACM*, 39(1):95–146, 1992.
- [21] R. Diaconescu. *Institution-independent Model Theory*. Birkhäuser, 2008.
- [22] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, K. G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3), 2018.
- [23] I. Incer. *The Algebra of Contracts*. Ph.D. thesis, University of California, Berkeley, 2022.
- [24] P. Cousot, R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *POPL*, 1977.
- [25] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, A. Schmitt. Combinators for bidirectional tree transformations: a linguistic approach to the view-update problem. *ACM TOPLAS*, 29(3), 2007.
- [26] K. Claessen, J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *ICFP*, 2000.
- [27] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, T. Gebru. Model cards for model reporting. *FAT**, 2019.
- [28] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, K. Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [29] Feast: an open source feature store for machine learning. <https://feast.dev>.
- [30] Tecton: a feature platform for machine learning. <https://www.tecton.ai>.
- [31] Hopsworks feature store. <https://www.hopsworks.ai>.
- [32] dbt: the analytics engineering workflow. dbt Labs. <https://www.getdbt.com>.
- [33] Cube: a semantic layer for data applications. <https://cube.dev>.
- [34] Malloy: an experimental language for data. <https://www.malloydata.dev>.
- [35] JPMorgan Chase & Co. *Report of JPMorgan Chase & Co. Management Task Force Regarding 2012 CIO Losses*. 16 January 2013.

- [36] United States Senate Permanent Subcommittee on Investigations. *JPMorgan Chase Whale Trades: A Case History of Derivatives Risks and Abuses*. 15 March 2013.
- [37] Autoriteit Persoonsgegevens. *Belastingdienst/Toeslagen: de verwerking van de nationaliteit van aanvragers van kinderopvangtoeslag*. 17 July 2020. The fining decision followed on 7 December 2021.
- [38] Parlementaire ondervragingscommissie Kinderopvangtoeslag. *Ongekend onrecht*. Tweede Kamer der Staten-Generaal, 17 December 2020.
- [39] Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency. *Supervisory Guidance on Model Risk Management*. SR 11-7 / OCC Bulletin 2011-12, 4 April 2011. The long-standing United States guidance; see Theorem 1.1.
- [40] Board of Governors of the Federal Reserve System, Federal Deposit Insurance Corporation, Office of the Comptroller of the Currency. *Supervisory Guidance on Model Risk Management*. SR 26-2 / OCC Bulletin 2026-13, 17 April 2026.
- [41] Prudential Regulation Authority, Bank of England. *Model Risk Management Principles for Banks*. Supervisory Statement SS1/23, May 2023 (amended April 2026).
- [42] European Parliament and Council. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence*, 2024.

© 2026 Ashutosh Sinha <ajsinha@gmail.com>. Licensed under Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International (CC BY-NC-ND 4.0), <https://creativecommons.org/licenses/by-nc-nd/4.0/>. Quoted supervisory guidance, regulation and scholarly work remain the property of their respective owners. This is a research document and is not legal, regulatory or financial advice.